Vectorized Dynamic Histograms for Sparse Oblique Forests

Anonymous Author(s)

Abstract

Sparse oblique (SPO), part of the top-ranked configuration of Google's Yggdrasil Decision Forests (YDF), improve the accuracy while maintaining interpretability of Random Forests (RF) and Gradient Boosted Trees (GBT) by scanning a sparse linear combination of features instead of a single feature. Because projections are sampled at runtime, training is significantly slower, as pre-run optimizations such as presorting cannot be used. We overhaul SPO training in YDF, first by fixing inefficiencies in the released version, achieving 2–5× speedup, then by devising two novel methods, one aimed at GBTs and the other at RFs: (1) Hierarchical AVX2 and AVX-512-vectorized histogram filling speeds up typical GBT-depth trees by up to 1.6× in SPO-GBT and 2× for SPO-RF, and (2) runtime-dynamic histograms between vectorized histograms and exact splits, speeds up deeper, RF depth-typical trees (depths 16 to purity) by up to 1.6× with no detectable effect on accuracy. Our optimizations bring SPO-RF training time on par with YDF's axis-aligned RF. We extensively evaluate on 8 natural and 11 synthetic datasets of up to 10.5 million rows and 1.6 million features, from depth 6 to purity, and open-source the implementation.

ACM Reference Format:
Anonymous Author(s). 2026. Vectorized Dynamic Histograms for Sparse Oblique Forests. In *Proceedings of* . ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Self-Assessment of Relevance

Experiments (primarily experimental contributions)

- Interplay between algorithmic parallelism, programming, and architecture
- Parallelism in machine learning
- Memory or I/O-efficient algorithms

This is an experimental study of training sparse oblique (SPO) decision forests, the top-ranked configuration of Google's Yggdrasil Decision Forests (YDF), on a 48-core CPU. Tree-level parallelism is already exploited; the cost is a single-threaded per-node kernel that gathers the projection's columns and bins or sorts them. Because the split feature exists only once the projection is drawn, presorting, global quantization and histogram subtraction do not apply: every projection is re-binned or re-sorted at every node, from millions of rows to a handful.

We contribute (1) a branch-free, in-register bin lookup (64 bins on AVX2, 256 on AVX-512) that lets histograms compete with vectorized sorting (Highway VQSort, our baseline), and (2) a per-node, size-based choice between the $O(n \log k + k)$ histogram and the $O(n \log n)$ exact finder, which no tree learner we know of does. We claim no new bounds or accuracy gains: a 2–5× stronger baseline than released YDF, then 1.3–2× at depth 6 and 1.2–1.6× from depth 16 to purity, accuracy-neutral on 36 datasets, on par with axis-aligned RF, open-sourced.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

,

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Gradient Boosted Trees (GBT) and Random Forests (RF) remain critical components in machine learning ensembles. Though tabular foundation models outperform tuned tree ensembles when datasets are smaller than 50k rows and 2000 features [13, 14, 17, 20, 35], GBTs perform best on high-dimensional datasets with millions to billions of rows [47]. RF offers several benefits to GBTs at the expense of inference latency. They are highly parallelizable, robust to hyperparameter selection [33] and give an unbiased error estimate and feature importances without holding out data, and adding trees does not overfit [4]. Due to their low computation cost, trees have a very strong cost-to-performance ratio, so are heavily used in resource usage-aware deployments [24].

Sparse oblique (SPO) splitting further improves the expressivity and accuracy of GBT and RF [39]. While RF selects a random $\sqrt{d}$ subset of features at each node and evaluates which feature best separates the data, sparse oblique forests sample $\sqrt{d}$ random projections, consisting of a sparse linear combination of features. The stochasticity and greater expressivity of the projections makes sparse oblique trees robust to noise and on average leads to more balanced trees [39], and its interpretability makes it a strong candidate for biomedical benchmarks [10].

Yggdrasil Decision Forests (YDF) [15] is Google's open-source implementation of decision forests, serving tens of millions of predictions per second at Google [15], and offers a wide variety of GBT and RF learners, including Sparse Oblique. YDF is integrated in Google's internal AutoML systems, where it is chosen in roughly a quarter of real-world use cases [15]. Sparse oblique splits are in YDF's top-ranked benchmark configuration for both RF and GBT, and ranks above every untuned XGBoost [8], LightGBM [19] and scikit-learn learner [30]. We include a similar comparison, adding CatBoost [34] (see Table 9). The YDF authors report that this configuration is significantly slower to train than their defaults ([15], Table 3).

Given the importance of SPO methods in YDF, we study how to reduce or eliminate training time overhead. Unlike a regular, feature axis-aligned (AA) decision tree, where the feature set remains constant throughout training, oblique random projections must be performed at runtime. At each node, projection features are randomly sampled, their linear combination is calculated, and the resulting new "feature" is scanned for the best split. There are combinatorially many such splits in the number of features. The evaluation of this larger feature space gives SPO its accuracy benefit, but precludes many optimization strategies such as pre-sorting features for exact splits [15, 45], histogram subtraction [8, 19], or oblivious trees [34].

A deep performance study of SPO-RF and SPO-GBT revealed bottlenecks during training. We contributed the fixes to YDF (Floyd's projections sampling, caching projection evaluator, a Fisher-Yates shuffle, among others) as a series of pull requests (see Appendix A.2 for full list). Independently, Google replaced `std::sort` with Highway VQSort [41], a highly optimized sorting library that heavily leverages SIMD instructions, which we adopted. Together these set a new baseline for this paper that is 2-5× faster than the open-source version with which we started. In this baseline, the majority of runtime is spent sorting nodes with *exact* splits: Highway sort is faster than every prior method, including all histogram variants and `std::sort` (Table 8). All speedups in this paper are measured against this stronger baseline. Against `std::sort` previously, dynamic selection alone achieves 2× (Appendix A.1).

Training time not only depends on tree depth, but is skewed to shallower or deeper levels depending on the splitting method: very large nodes near the root disproportionately affect the $O(n \log n)$ sorting cost, while large numbers of small nodes punish the $O(k)$ startup cost in histogram's $O(n \log k + k)$ complexity. GBTs are trained to depth 6, often to leverage a fast inference engine, such as QuickScorer [22], which has a limit on maximum number of nodes. Training to depth 16-24 is common for Random Forests, and training to purity (full depth) is useful for posterior estimates [10, 11]. The two families thus stress opposite ends of the depth range, and we contribute one optimization for each. Based on this observation, we contribute:

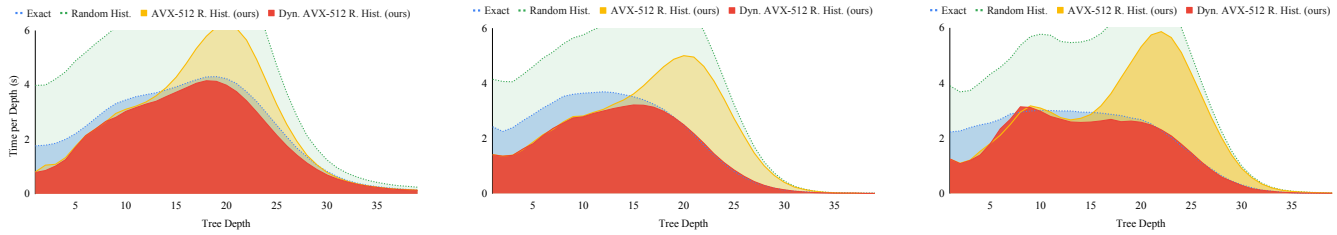
(1) Vectorization of histogram construction. Histogram construction is asymptotically more efficient than sorting and should be faster when there are a sufficient number of samples. Yet with the powerful Highway sort replacing `std::sort`, the baseline histogram implementation using `std::upper_bound()` is much slower than vectorized Highway sort (Figure 1). We propose a hierarchical vectorized histogram filling implementation that leverages AVX2 and AVX-512, and delivers up to 2× speedup over the SPO-RF Exact and 1.6× over SPO-GBT, and beats scalar Random Hist. at all tree levels, while being mathematically identical.

(2) Runtime-adaptive histograms. In deeper trees, the number of active nodes grows near-exponentially, and peaks at around halfway to purity. Though these nodes have few samples in them, histogram approaches must still pay the $O(k)$ startup cost of bin generation and scanning that Exact avoids (Fig. 1). We propose a Dynamic switching method that adaptively chooses between vectorized histograms and exact splits depending on the number of samples at each node, with no detectable effect on accuracy (Table 9). This achieves up to 1.6× speedup when training deeper trees, bringing SPO performance on par with Axis-Aligned in YDF (Table 3).

Neither optimization is tied to a depth or a model family. Vectorized filling accelerates every histogram split, and the dynamic switch acts on node size rather than depth, so both apply to GBT and RF alike, and to axis-aligned splits whenever features are not pre-sorted (Figure 1(c)). At depth 6 over millions of rows no node is small enough to switch, so dynamic selection reduces to vectorized histograms, and at purity, all nodes switch. We therefore report each family in the setting it is used in practice, GBT at depth 6 and RF from depth 16 to purity. We provide extensive results on a variety of both natural and synthetic datasets (Table 5), we sweep the full depth range in Table 6. In addition, we provide a heatmap of predicted speedups by dataset size based on our evaluation (Figure 7). An open-source implementation is available at (anonymized).

2 Related Work

Oblique decision trees split on linear combinations of features rather than on a single feature. Breiman's original random forest paper proposed Forest-RC, which draws random linear combinations of a few inputs at each node [4]. OC1 searches for hyperplanes by



(a) SPO-GBT **(b) SPO-RF** **(c) AA-RF**

Figure 1: Training time by tree depth on YouTube-8M, one panel per model family: SPO-GBT, SPO-RF and axis-aligned RF (AA-RF). Each panel compares the existing Exact and Random Histogram (256-bins) splitting with our Vectorized and Dynamic-Vectorized (256-bin) methods.

hill climbing with random perturbations [27], canonical correlation forests derive the projection at each node from CCA [36], and TAO learns sparse oblique trees by alternating optimization over the node hyperplanes [6], with bivariate trees as the two-feature special case [18]. Sparse projection oblique random forests (SPO-RF) [39] sample sparse random projections at each node and are implemented in treeple [21] and in YDF [15], whose sparse oblique splits we optimize. SPO-RF is also the base learner of MIGHT, which trains forests to purity to obtain calibrated posteriors for biomedical screening [10, 11]. Common to all of these methods is the fact that the feature being split exists only once the node's projection has been drawn and therefore cannot be presorted/pre-quantized before training.

Axis-aligned learners find splits either exactly or with histograms. Exact splitting sorts the candidate feature's values within the node or scans a pre-sorted index of the whole column: ranger [45], XGBoost's exact method [8] and YDF [15] pre-sort, and scikit-learn's forests sort in the node [30]. Histogram splitting, the default in LightGBM [19], CatBoost [34], XGBoost's histogram method [8], TF Boosted Trees [32] and Spark MLlib [26], quantizes every feature once before training into a fixed number of bins, so that filling a node's histogram is a scatter-add over stored bin indices with no search in the loop. XGBoost's local proposal variant [8] and YDF's histogram splits [15] instead recompute bin boundaries per node, from the node's quantiles, its value range, or at random. Further savings come from sharing work across nodes: histogram subtraction derives one child's histogram from the parent's [19], CatBoost's oblivious trees split every node of a level on the same feature and bin grid [34], and GPU trainers assemble partial histograms per thread block [42, 46].

Resolving a key against a register of boundaries with one compare and a population count is established: per tree level in k-ary search [38] and in cache-resident range indexes [31], and in PANDA as a two-level SIMD scan over histogram boundaries [29]. All three build their boundaries once, then bin everything against them. Sparse oblique splits draw new boundaries for every projection at every node, so the structure is rebuilt per histogram and may serve only a few hundred lookups. Our lookup is sized for this: $B = W^2$ bins, two compare-and-count steps, one extra register, and an index, mathematically identical to `std::upper_bound` (Section 3.2).

To our knowledge, no tree learner switches between exact and histogram splitting per node, which is the choice our dynamic histograms make.

Highway [41] is a C++ library of portable SIMD operations with runtime dispatch across x86, ARM and RISC-V targets. Its vectorized quicksort, VQSort, is the sort behind YDF's exact splits and the baseline for every speedup in this paper.

Inference performance optimization of RF and GBT includes: trees being compiled into C++ [9, 16] to improve memory layout and reduce branching and into tensor operations [28]. Cache-efficient memory layouts have been applied to nodes in multiple trees to reduce random I/O [5, 23].

3 Method

We describe the per-node computation of sparse oblique splitting and its cost, then the two optimizations: a vectorized bin lookup (Section 3.2) and a per-node choice between histogram and exact splitting (Section 3.3). Throughout, n is the number of active samples at a node, d the number of features, P the projections per node, k the histogram bins and w the lanes of a vector register.

3.1 Sparse Oblique Workflow and Its Cost

YDF stores the data column by column and never copies it; a node is a sorted list of row indices, and each split partitions the parent's list in place. For RF the root holds a bootstrap sample of n rows drawn with replacement, about 63% of them distinct. At each node SPO draws $P = \lceil \sqrt{d} \rceil$ projections, capped at 1000 [39], each a sparse vector with Binomial$(d,\ 1.5/d)$ non-zeros, i.e. 1.5 on average, features chosen without replacement and weights uniform in $[-1, 1]$. Every projection passes through four stages (Figure 2):

(1) *Sample Projection*: draw the non-zero features and weights.
(2) *Build Projected Feature*: gather the selected columns at the active rows and accumulate the weighted sum into n values. Sparse in both dimensions and increasingly scattered with depth.
(3) *Build Histograms*, or sort: bin each value and count its class, or sort the values for exact splitting.
(4) *Evaluate Split*: score every candidate threshold by information gain. Keep the best over all P projections.

Exact splitting packs each (value, label) pair into a 64-bit key, sorts the n keys with Highway's VQSort [41] and scans them once,

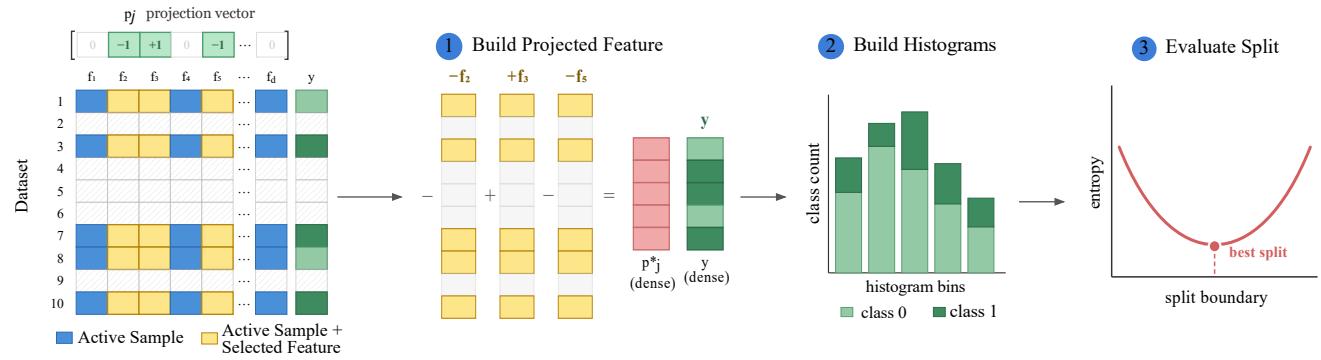


Figure 2: Workflow at each tree node. Histogram splitting of a random linear combination of features requires sparse access in both rows and columns, computing a vector sum, building histograms and evaluating split boundaries.

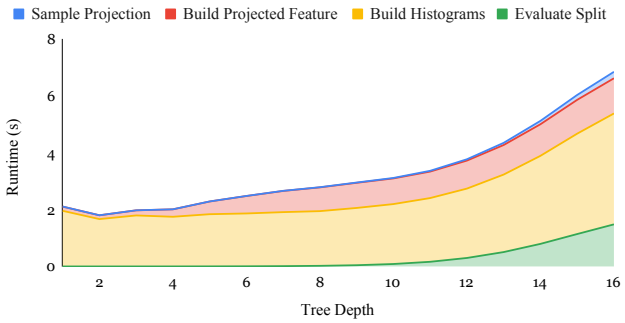


Figure 3: Comparative runtime of the different components of computation for histogram splitting. The dataset has 1M samples and 4096 features.

considering the midpoint between every pair of adjacent distinct values: $O(n \log n)$ per projection. *Random Histogram* splitting draws k thresholds uniformly between the node's minimum and maximum value, sorts them, bins each value by binary search over the thresholds, and scans the k candidates: $O(n \log k)$ for the lookups plus an $O(k)$ term independent of n for drawing and sorting the thresholds, initializing the counters and scanning the bins. Because the thresholds are random rather than equally spaced, the bin index has no closed form, unlike learners that quantize each feature once before training [19, 34]. The search runs once per value, per projection, per node.

Figure 3, from our per-node timers, breaks Random Histogram training time down by stage and depth on 1M rows and 4096 features. Building histograms dominates at every depth. Its cost per depth is flat while nodes are large, since each depth bins all n samples once per projection, and grows from depth 10 on, where the number of nodes doubles per level and the $O(k)$ work per projection grows with it. Evaluate Split, also $O(k)$ per projection, appears at the same depth. The two optimizations follow: in shallow trees the per-value lookup dominates, in deep trees the per-projection fixed cost does.

3.2 Vectorization of Histogram Filling

YDF bins a value with `std::upper_bound`, a binary search over the k sorted thresholds (Figure 4, left): $\log_2 k$ dependent steps, each ending in a data-dependent branch that can cause mispredictions. A linear scan avoids the branches but costs k comparisons per value.

We replace the search by two levels of SIMD comparisons (Figure 4 right). With w lanes, $w = 8$ for AVX2 and $w = 16$ for AVX-512, we use $k = w^2$ bins. Once per histogram we copy the sorted thresholds $T[0, k)$ into an aligned array and build a coarse table $C[0, w)$ of the last threshold of each group of w bins, $C[g] = T[w(g + 1) - 1]$. Per value x, one broadcast, one compare against C and one `popcnt` give K, the number of groups whose thresholds are all at most x, i.e. the group containing x; a second compare against the w thresholds of group K and a second `popcnt` give the count j of thresholds at most x within it, and the bin is $wK + j - 1$, or -1 if x lies below every threshold. If $K = w$ the value belongs to the last bin.

The lookup is seven vector and bit-count instructions on AVX2 and five on AVX-512, whose compares write a mask register directly, with no data-dependent branch. Thresholds are drawn, sorted and used exactly as before, and the lookup returns the same index as `std::upper_bound` for every input, which debug builds assert, so the trees are bit-identical to the scalar implementation. The kernel is selected at run time from the CPU and the bin count, with a scalar fallback, and serves the axis-aligned histogram finder unchanged. The two widths are not interchangeable: the lookup costs the same, but the $O(k)$ per-projection work is four times larger with 256 bins, so AVX-512 wins only where n is large relative to k and loses in deep trees (Figure 8).

3.3 Dynamic Histogramming

Histogram and exact splitting are expensive at opposite ends of the tree (Figure 1). Near the root the $O(n \log n)$ sort dominates and histograms are faster [15, 34], and deep in the tree the number of nodes doubles per level while samples per node halve, and the $O(k)$ work per histogram, paid at every node regardless of n, dominates.

We therefore choose the finder per node: before the projection loop, a node with $n < \tau$ samples uses exact splitting for all its projections, otherwise vectorized histograms. The choice costs one



Figure 4: Mapping sample points to histograms using binary search (left) and our vectorized AVX2 implementation (right).

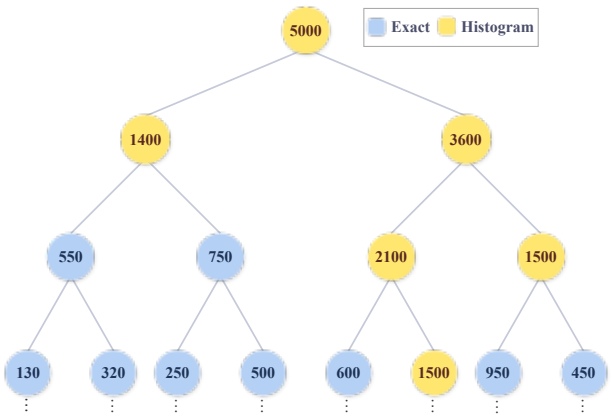


Figure 5: Illustration of the selection of exact splitting and histogram splitting as a function of the number of active samples in a tree node with a breakeven point of 1300.

comparison per node, and nodes at the same depth may have differerent split types (Figure 5). The threshold τ is a single integer hyperparameter that depends on the kernel and the sort, not on the dataset. We provide a microbenchmark to heuristically set it at the first run on a new machine. For our test, we chose $\tau = 250$ for the 64-bin AVX2 kernel and 1000 for the 256-bin AVX-512 kernel. A full runtime comparison of various thresholds is provided in Figure 9. Near the top of the tree, most nodes clear the threshold and dynamic splitting chooses as vectorized histograms, whereas below it a growing share of nodes is exact, avoiding the histogram's fixed cost.

Dynamic forests are not node-for-node identical to histogram forests: exact splitting considers every midpoint instead of k random thresholds, and since the histogram finder consumes random numbers for its thresholds and the exact finder does not, projections drawn after the first switch differ as well. The split rule is unchanged at every node with $n \geq \tau$, and we find no detectable effect on accuracy (Table 9).

4 Results

We report each contribution in the setting described in the introduction. Section 4.1 sets the protocol and the baseline. Section 4.2 measures vectorized histogram filling on depth-6 trees, for SPO-GBT and SPO-RF. Section 4.3 follows SPO-RF from depth 16 to 24, where the dynamic switch takes over from vectorization, and Section 4.4 completes the depth range at purity and compares against YDF's axis-aligned RF. Section 4.5 maps the speedup over rows and features, Section 4.6 separates the two optimizations and the two bin counts, Section 4.7 varies the switch threshold, and Section 4.8 shows that this does not change accuracy. Table 6 (SPO-RF, six depths) and Table 7 (SPO-GBT, depth 6) in the appendix hold the full timing results.

4.1 Experimental Setup

Hardware and software. All timings come from one AWS m7i.metal-24xl instance (Intel Xeon Platinum 8488C, 48 physical cores, 384 GiB, Ubuntu 24.04) running YDF compiled with Intel ICX 2025.2.1 and AVX-512 enabled, with 48 threads. Random forests train one tree per thread; gradient-boosted trees parallelize the split search within each tree. Times cover training only: dataset loading and model serialization are excluded.

Dataset	Task	Rows	Feat.
HIGGS [43]	bin	11,000,000	28
SUSY [44]	bin	5,000,000	18
Epsilon [7]	bin	400,000	2,000
YouTube-8M [1]	mul*	3,888,919	1152
Shifts Weather [25]	reg*	3,129,592	127
ClimSim [48]	reg*	10,091,520	124
Jane Street [12]	reg*	10,213,190†	82

Table 1: Natural Datasets used in our experiments. * - treated as `bin`. † - subset used. Full datasets and their description in Table 5.

Learners. SPO-RF: 240 trees, bootstrap sampling with replacement and a bag size equal to the number of rows, `min_examples=1`, maximum depth 6, 10, 16, 20, 24 or unlimited (purity). SPO-GBT: 300 trees of depth 6, shrinkage 0.1, `min_examples=5`. Every node samples $\lceil\sqrt{F}\rceil$ sparse projections (at most 1000) with 1.5 expected non-zero features each. YDF defaults apply elsewhere.

Split finders and baseline. Exact splits sort the projected values with Highway VQSort and are the baseline for every speedup below: speedup = $T_{\text{Exact}}/T_{\text{method}}$, so values above 1 are faster. Table 8 in the appendix relates this baseline to the `std::sort` exact finder of the released YDF, which it improves by 1.8–3.5×. Random histograms draw their bin boundaries per projection and use 64 bins with the scalar (`std::upper_bound`) and AVX2 binners and 256 bins with the scalar and AVX-512 binners. The dynamic variants bin nodes with at least τ rows and sort the rest, with $\tau = 250$ for AVX2/64 bins and $\tau = 1000$ for AVX-512/256 bins (Section 4.7). The scalar 64-bin binner has a breakeven threshold $\tau > 50000$, and therefore appears only as an non-Dynamic baseline.

Datasets and protocol. Sevel natural datasets (Table 1) and 13 Trunk shapes spanning 15k to 4.5M rows and 32 to 1.6M features (Table 6). Because regression is slightly slower than classification, regression targets are thresholded at the training-set median and the multi-label YouTube-8M target is its most frequent class against the rest. Accuracy protocols are stated in Section 4.8.

4.2 Depth 6 - GBTs

Depth-6 trees over millions of rows have no small nodes: on the seven large natural datasets no node falls below the switch threshold, so no dynamic versions were included (Table 6).

SPO-GBT. With the AVX2 64-bin binner, SPO-GBT trains 1.3–1.6× faster than Exact on the seven large natural datasets (Tables 7, 2). HIGGS and Epsilon 1.3×, SUSY and Shifts weather 1.4×, YouTube-8M 1.5×, ClimSim and Jane Street 1.6×). The AVX-512 256-bin is within 5% of it on every dataset (Figure 6, SPO-GBT panel). The YDF baseline Random Hist. scalar is slower than Exact at this depth on every dataset (0.6–0.9× with 64 bins, 0.4–0.6× with 256 bins). Vectorization of histograms flips this: the vectorized binner is 1.5–2.2× (64 bins) and 1.7–2.9× (256 bins) faster than its scalar counterpart on the natural datasets while building the same trees. On the Trunk shapes the gain over Exact is 1.2–1.4× from 150k rows up and vanishes on the two 15k-row shapes (1.0×), where the total work (proportional to n. rows) is also low.

SPO-RF. The same kernel gives 1.6–2.0× on the seven large natural datasets at depth 6 (Tables 6„ 2). Epsilon 1.6×, Shifts weather 1.7×, HIGGS and SUSY 1.8×, YouTube-8M 1.9×, ClimSim and Jane Street 2.0×) and 1.4–1.9× on the Trunk shapes, rising with the number of rows. Scalar histograms are again unreliable against Exact (0.7–1.6×), whereas the vectorized ones beat Exact on every dataset. How SPO-GBT and SPO-RF compare with axis-aligned YDF and with XGBoost, LightGBM and CatBoost, which pre-sort or pre-quantize their fixed feature set, is reported in Table 3.

4.3 Depth 16 - RF, Dynamic

Between depth 16 and depth 24 the need for dynamic switching starts to appear (Table 6, Figure 6).

Depth 16. Vectorized dynamic histograms (AVX2, 64 bins) train 1.3–1.6× faster than Exact on the seven large natural datasets (Epsilon 1.3×, Shifts weather, HIGGS and YouTube-8M 1.4×, SUSY and Jane Street 1.5×, ClimSim 1.6×) and 1.2–1.4× on the Trunk shapes. The nodes of a 10M-row dataset still hold hundreds of rows at this depth, so on the natural datasets the switch fires rarely and vectorization supplies almost all of the gain: the dynamic and vectorized-only variants differ by at most 8% (Epsilon). On the Trunk shapes with 15k–150k rows the switch already matters: vectorized-only histograms fall to 0.95–1.2× and the switch lifts them to 1.2–1.3×.

Depths 20 and 24. Each level doubles the number of nodes and halves their size, and the per-projection cost of drawing and scanning the bins (Figure 8) outweighs sorting < 100 rows. Vectorized-only histograms fall to 0.9–1.5× Exact on the natural datasets at depth 24 (Epsilon 0.9×, HIGGS 1.2×, ClimSim 1.5×) and to 0.9–1.1× on the Trunk shapes, with 256 bins they fall to 0.6–1.4× and 0.5–0.8×. The switch keeps 1.2–1.5× on the natural datasets at depth 24 (Epsilon 1.2×, HIGGS, Shifts weather and YouTube-8M 1.3×, Jane Street and SUSY 1.4×, ClimSim 1.5×) and 1.1–1.3× on every Trunk shape, with the two bin counts within 2% of each other.

4.4 Training to Purity

At purity the trend completes (Tables 6, 2), Figure 6, bottom-right). Histogram-only methods now lose to Exact on most datasets: scalar 64-bin histograms run at 0.6–1.0× Exact on the natural datasets and 0.6–0.75× on Trunk, vectorized 64-bin at 0.8–1.4× and 0.9–1.2× (HIGGS 0.95×, Jane Street 0.98×, Epsilon 0.90×. Only ClimSim, Shifts weather and YouTube-8M stay above 1.1×), and AVX-512 256-bin at 0.5–1.2×. Dynamic switching makes the vectorized histogram a net win on all 21 datasets: 1.2–1.5× on the seven large natural datasets (Epsilon, HIGGS and Jane Street 1.2×; SUSY, Shifts weather and YouTube-8M 1.3×; ClimSim 1.5×) and 1.1–1.3× on Trunk. Relative to vectorized-only histograms the switch is worth 1.06–1.31× on the natural datasets (HIGGS 1.29×, Epsilon 1.31×) and 1.14–1.32× on Trunk with 64 bins, and 1.3–2.2× with 256 bins, where it removes the whole bin-count penalty.

Parity with axis-aligned RF. Table 3 compares full-depth forests on HIGGS, SUSY and Epsilon with YDF's axis-aligned RF, which pre-sorts its fixed features. The Exact SPO-RF baseline is 1.13–1.24×

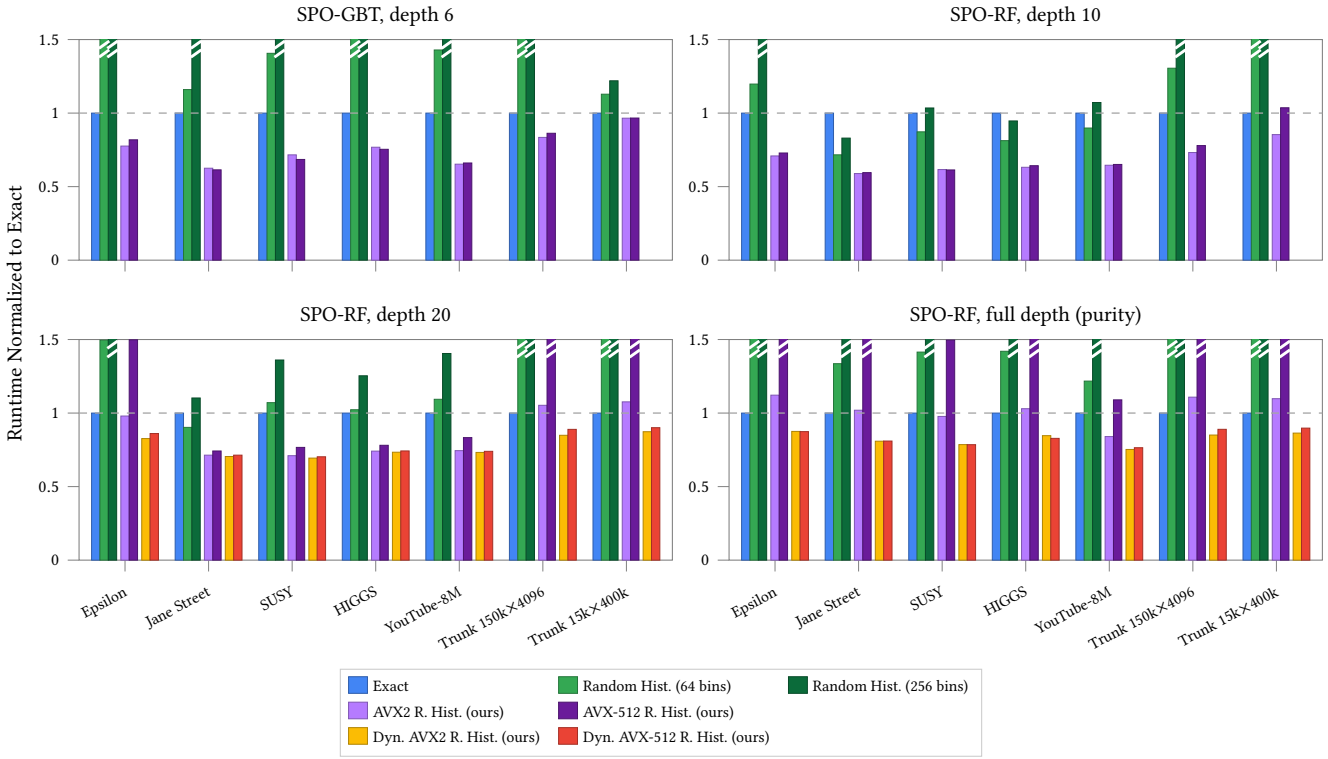


Figure 6: End-to-end training time on select datasets, normalized to the performance of SO-YDF with exact splits. Panels: SPO-GBT at depth 6; SPO-RF at depth 10, depth 20 and full depth (purity). Blocks of each cluster, left to right: Exact and scalar random histograms; our vectorized histograms; our dynamic-vectorized histograms (not run at depths 6 and 10). Bars above 1.5 are cut off (white break mark).

Dataset	SPO-GBT, depth 6		SPO-RF, depth 6		SPO-RF, depth 10		SPO-RF, depth 20		SPO-RF, purity	
	Exact	Ours	Exact	Ours	Exact	Ours	Exact	Ours	Exact	Ours
HIGGS 10.5M×28	1619	1243 (1.3×)	79.6	44.7 (1.8×)	141	90.1 (1.6×)	249	183 (1.4×)	364	298 (1.2×)
SUSY 4.5M×18	638	456 (1.4×)	27.0	14.9 (1.8×)	46.1	28.4 (1.6×)	80.6	55.9 (1.4×)	118	90.9 (1.3×)
Epsilon 400k×2000	96.5	74.8 (1.3×)	12.0	7.4 (1.6×)	23.7	16.8 (1.4×)	51.0	42.1 (1.2×)	59.9	50.8 (1.2×)
YouTube-8M 3.9M×1152	1021	666 (1.5×)	137	70.9 (1.9×)	235	151 (1.6×)	438	320 (1.4×)	504	379 (1.3×)
Shifts weather 3.1M×127	491	353 (1.4×)	34.3	20.0 (1.7×)	58.1	39.4 (1.5×)	92.2	68.9 (1.3×)	93.9	72.0 (1.3×)
ClimSim 10.1M×124	1741	1118 (1.6×)	146	73.2 (2.0×)	228	128 (1.8×)	298	189 (1.6×)	308	203 (1.5×)
Jane Street 10.2M×82	2286	1429 (1.6×)	121	60.1 (2.0×)	202	119 (1.7×)	374	264 (1.4×)	593	479 (1.2×)

Table 2: End-to-end training time (s) on the natural datasets: Exact (HWY) vs. ours (Vec. Random Hist. AVX2, 64 bins for SPO-GBT; Vec. Dynamic Hist. AVX2, 64 bins for SPO-RF). Speedup over Exact in parentheses. All depths and arms in Tables 6 and 7.

slower than AA-RF (363 vs. 292 s on HIGGS, 120 vs. 104 s on SUSY, 59 vs. 52 s on Epsilon). Vectorized dynamic histograms close the gap: 296 s (AVX2) and 288 s (AVX-512) on HIGGS against 292 s, and 3–16% faster than AA-RF on SUSY (92 and 90 s vs. 104 s) and Epsilon (50.5 and 50.7 s vs. 52.1 s). Sparse oblique forests thus no longer cost training time over axis-aligned ones in YDF, at the accuracy of Section 4.8.

4.5 Dataset Scalability

Figure 7 places every dataset by rows and features and colors it by the speedup of vectorized dynamic histograms (AVX2, 64 bins) over Exact at depth 6, depth 16 and purity; the shading interpolates between the measured points and predicts the speedup for a dataset of a given size. Two regularities of Table 6 shape the map. First, the speedup grows with the number of rows at every depth. On the Trunk shapes at depth 6 it rises from 1.4× at 15k rows through 1.6× at 150k and 1.6–1.7× at 1–1.5M to 1.9× at 4.5M, and the 10M-row natural datasets reach 1.8–2.0×; at purity the same progression reads 1.1×, 1.2×, 1.2–1.3×, 1.25× and 1.2–1.5×. Second, the number of features barely matters: with 150k rows the depth-6 speedup is 1.56, 1.56, 1.55 and 1.60× at 4096, 40k, 160k and 400k features, and with 1.5M rows 1.71, 1.71 and 1.69× at 4096, 16384 and 40k. The feature count scales the projection sampling and evaluation, which every split finder pays alike, whereas the choice between

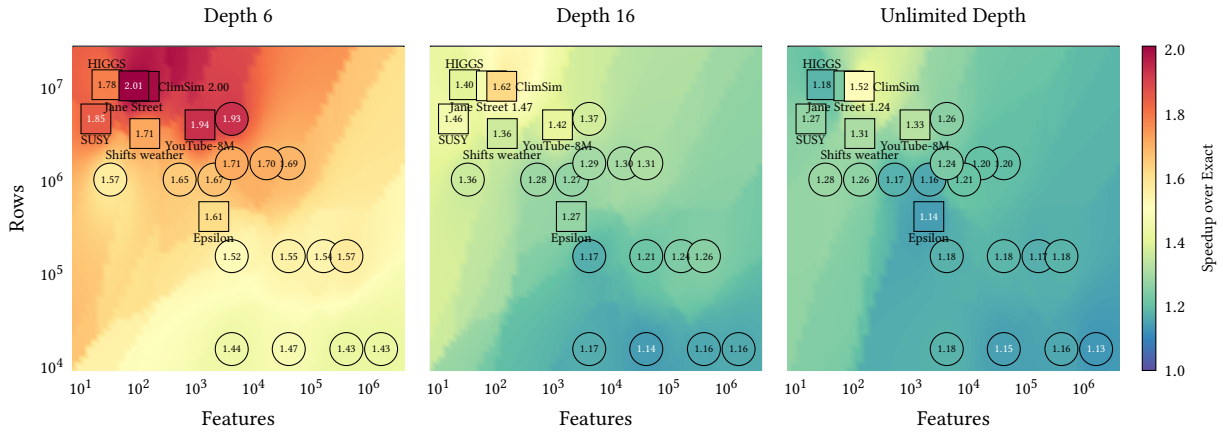


Figure 7: Speedup heatmap by dataset dimensions, simulated using Trunk synthetic data (circles), of our SPO-RF Dyn-Vec method over Exact. Also added are natural datasets HIGGS, SUSY, Epsilon, YouTube-8M, Shifts weather, ClimSim and Jane Street (squares). Each marker is labelled with its measured speedup. The background shading interpolates the speedup by inverse-distance weighting: every point averages its three nearest measured points, weighted by the inverse square of its distance.

histogram and sort acts on the n projected values of a node. The widest shapes, 15k×400k and 15k×1.6M, are the least favourable (1.4× at depth 6, 1.1× at purity) because their split search is a minor share of the runtime (Section 4.2).

4.6 Ablation - Dynamic Vectorized Histograms

Table 6 contains the two single-change comparisons that separate the contributions, and the two bin counts side by side.

Vectorization alone. The scalar and vectorized binners with the same bin count build the same histograms and the same trees, so their ratio is a pure kernel speedup. On the natural datasets it is 1.2–1.8× (64 bins, AVX2) and 1.3–2.2× (256 bins, AVX-512) at depth 6 and 1.3–1.5× and 1.3–1.6× at purity; on Trunk 1.4–2.1× and 1.4–2.6×. It never drops below 1.2× at any depth on any dataset. Vectorized histograms therefore beat scalar ones at every tree level; whether they beat Exact depends on the depth (Sections 4.2–4.4).

Dynamic switching alone. With the binner held fixed, on the seven large natural datasets the switch changes nothing at depths 6 and 10 (within 3%), adds up to 8% at depth 16, 3–28% at depth 24 (HIGGS 5%, SUSY 8%, Epsilon 28%) and 6–31% at purity with 64 bins. With 256 bins, whose all-histogram variant carries four times the bin setup and scan, it adds 1.15–2.1× at depth 24 and 1.3–2.2× at purity. Where every node has hundreds of rows the switch is idle and costs nothing; where nodes are small it recovers what histograms lose.

4.7 Dynamic Threshold Sensitivity Study

Figure 9 varies the switch threshold τ from 100 to 10000 rows on HIGGS, SUSY, Epsilon and YouTube-8M at full depth. The optimum lies at 300–500 rows for AVX2 with 64 bins (1.22–1.34×) and at 800–1200 rows for AVX-512 with 256 bins (1.19–1.32×): the larger bin count raises the fixed cost per histogram and with it the node size at which a histogram beats a sort. The curves are flat. Every AVX2 curve stays within 2% of its maximum between 200 and 1000 rows and every AVX-512 curve between 400 and 2500; pushing τ to 10000, which sorts all but the largest nodes, still retains 93–98% of the best speedup, and $\tau = 100$ costs 1–2% (AVX2) and 5–10% (AVX-512). The fixed values used throughout, $\tau = 250$ (AVX2) and $\tau = 1000$ (AVX-512), are within 1% of every per-dataset optimum, so no per-machine tuning was needed for the results of this section.

4.8 Accuracy

Vectorization changes no model: the scalar and vectorized binners bin identically and produce identical forests, which is why their accuracy columns coincide in Table 9. The dynamic switch does change the model, since nodes below τ take an exact split where an all-histogram forest would take a binned one. Table 4 measures the effect on held-out accuracy against Exact on 34 OpenML CC18 binary tasks (10-fold cross-validation × 3 seeds) plus HIGGS and SUSY. For SPO-RF the mean difference of the dynamic forest to Exact is $+5 \times 10^{-3}$ with a 95% interval of $[-4, +14] \times 10^{-3}$ that includes zero; it wins on 13 datasets, ties on 17 and loses on 6, and only two per-dataset differences survive Holm correction. For SPO-GBT the difference is $+1 \times 10^{-3}$ $[-1, +3]$ (9 wins, 20 ties, 7 losses), with electricity and adult significantly below Exact by 1.6 and 0.1 points. The dynamic forests are non-inferior to Exact within 2.5×10^{-3} (RF) and 1×10^{-3} (GBT) accuracy at $\alpha = 0.05$, and the all-histogram forests fall in the same intervals. On the large datasets the SPO-RF finders agree to three decimals (HIGGS 0.762 and SUSY 0.802 for Exact and Dynamic alike) and the SPO-GBT ones within 0.2 points. Against XGBoost, LightGBM, CatBoost and axis-aligned YDF (Table 3, 9), the RF and GBT learners are very similar, and the dynamic vectorized configuration ties the exact baseline on 34 of 36 datasets.

Table 3: Training time comparison to popular libraries using GBT (Boosting) and Random Forests (bagging). These are not like-for-like comparisons (see footnote), but are included in expectation of high interest in such a comparison.

Random Forest (full-depth)

Dataset	SPO-RF Baselines		SPO-RF Our Methods				Other RF			
	Exact	Random Hist.	Vec. Random Hist.	Vec. Random Hist.	Vec. Dynamic Hist.	Vec. Dynamic Hist.	AA-RF	XGBoost†	LightGBM†	CatBoost†
	HWY	scalar, 64 bins	AVX2, 64 bins	AVX-512, 256 bins	AVX2, 64 bins & HWY	AVX-512, 256 bins & HWY	Exact	RF mode	RF mode	RF mode
HIGGS	363.0	524.2	382.3	560.1	296.2	**287.5**	292.4	3000.9	2598.8	—
SUSY	119.6	171.3	119.4	175.6	91.8	**90.2**	104.3	1048.6	2281.1	—
Epsilon	59.1	95.1	68.6	109.8	**50.5**	50.7	52.1	OOM	OOM	—

Gradient Boosted Trees (depth 6)

Dataset	SPO-GBT Baselines		SPO-GBT Our Methods				Other GBT			
	Exact	Random Hist.	Vec. Random Hist.	Vec. Random Hist.	Vec. Dynamic Hist.	Vec. Dynamic Hist.	AA-GBT*	XGBoost*	LightGBM*	CatBoost*
	HWY	scalar, 64 bins	AVX2, 64 bins	AVX-512, 256 bins	AVX2, 64 bins & HWY	AVX-512, 256 bins & HWY	Exact			
HIGGS	1617.9	2519.0	1242.3	1225.8	—	—	746.6	21.8	**18.2**	35.9
SUSY	638.2	896.7	460.7	437.9	—	—	267.6	7.1	**6.1**	14.7
Epsilon	96.6	159.4	75.2	74.8	—	—	490.0	56.1	52.7	**24.0**

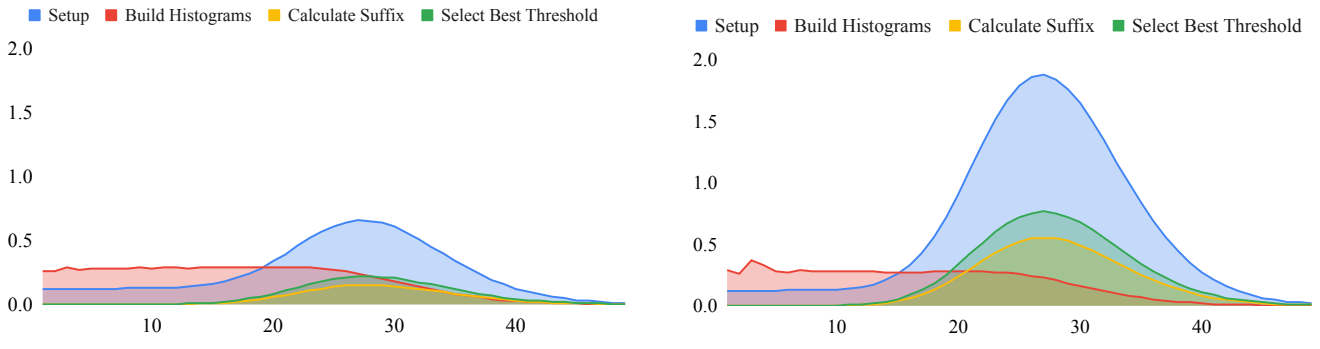
† RF implementations of this method are not directly comparable. XGBoost 3.4.1 warns that `XGBRFClassifier` is deprecated and "does not implement a conventional random forest", and LightGBM only parallelizes within a tree, similar to YDF-GBT, even in Random Forest mode. This explains the slow performance of the two methods. CatBoost does not support Random Forest (Bagged) mode.
* These methods benefit from features being fixed throughout training (no random projections), and can use optimizations like pre-sorting. It is unclear to us why YDF's AA-GBT does not benefit to a similar extent.

Table 4: Held-out test accuracy of random-histogram split finding vs. exact splits, aggregated over 36 datasets (34 OpenML CC18 binary tasks, 10-fold CV × 3 seeds + HIGGS and SUSY). Δ = method − Exact in units of 10^{-3} accuracy; CIs are per-dataset paired t intervals. W/T/L counts Wins/Ties/Losses ($|\Delta| \le 10^{-3}$ is a tie). LOO Δ = leave-one-out, dropping the most influential dataset. Margin = smallest one-sided non-inferiority margin passing at α=0.05; Sig. = datasets with a Holm-corrected per-dataset Wilcoxon difference.

Learner	Split finder	n	Δ [95% CI] ($\times 10^{-3}$)	LOO Δ Acc. [95% CI]	W/T/L	Margin ($\times 10^{-3}$)	Sig.
SPO-RF	Dynamic (64 bins, τ=250)	36	+5.2 [-3.7, +14.2]	+0.8 [+0.0, +1.7]	13/17/6	2.5	2
	Random, 64 bins	36	+6.5 [-5.6, +18.5]	+0.6 [-0.4, +1.6]	12/19/5	5	2
	Random, 256 bins	34	+7.3 [-5.4, +20.1]	+1.1 [-0.0, +2.3]	12/15/7	5	0
SPO-GBT	Dynamic (64 bins, τ=250)	36	+1.0 [-1.0, +3.1]	+0.4 [-1.2, +2.0]	9/20/7	1	2
	Random, 64 bins	36	+1.1 [-0.8, +3.0]	+0.6 [-1.1, +2.2]	13/15/8	1	2
	Random, 256 bins	34	+0.7 [-2.0, +3.3]	-0.5 [-1.7, +0.7]	10/12/12	2.5	0

5 Conclusions and Discussion

We overhaul sparse oblique (SPO) training in YDF. Fixing inefficiencies in the released implementation and adopting Highway VQSort raised the baseline by 2–5×, and against it we contribute two novel methods. AVX2 and AVX-512 vectorized histogram filling speeds up depth-6 SPO-GBT by 1.3–1.6× and SPO-RF by up to 2× on the large natural datasets, where scalar histograms lose to sorting. Dynamic switching between vectorized histograms and exact splits by node



(a) AVX2 (64 bins) (b) AVX-512 (256 bins)

Figure 8: Vectorized histogram cost breakdown: AVX2 with 64 bins (left) and AVX-512 with 256 bins (right).

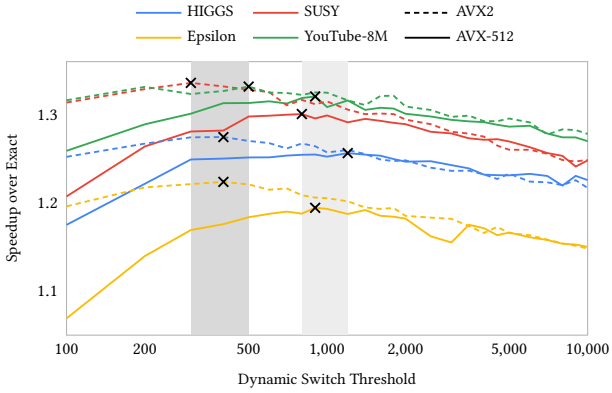


Figure 9: Full-depth train speedup over Exact by a linesearch over Dynamic Thresholds. Above the threshold, Random-Vectorized is used, and below, Exact. Shaded bands: the range of the AVX-512 maxima (light, 800–1200 rows) and of the AVX2 maxima (dark, 300–500 rows).

size retains 1.2–1.6× from depth 16 to purity, where histogram-only methods fall behind Exact, with no detectable effect on accuracy on 36 datasets. Together they bring SPO-RF training time on par with YDF's axis-aligned RF. Both act on node size rather than depth or model family, so they extend to axis-aligned splits without pre-sorting; batching the many small nodes of deep trees into single GPU kernels is future work.

References

[1] Sami Abu-El-Haija, Nisarg Kothari, Joonseok Lee, Paul Natsev, George Toderici, Balakrishnan Varadarajan, and Sudheendra Vijayanarasimhan. Youtube-8m: A large-scale video classification benchmark. *arXiv preprint arXiv:1609.08675*, 2016.

[2] Jon Bentley and Bob Floyd. Programming pearls: a sample of brilliance. *Commun. ACM*, 30(9):754–757, September 1987.

[3] Bernd Bischl, Giuseppe Casalicchio, Matthias Feurer, Pieter Gijsbers, Frank Hutter, Michel Lang, Rafael G Mantovani, Jan N van Rijn, and Joaquin Vanschoren. Openml benchmarking suites. *arXiv preprint arXiv:1708.03731*, 2017.

[4] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[5] James Browne, Tyler M. Tomita, Disa Mhembere, Joshua T. Vogelstein, and Randal C. Burns. Forest packing: Fast parallel decision forests. *IEEE Transactions on Computers*, 71(1):73–86, Jan 2022.

[6] Miguel A Carreira-Perpinán and Pooya Tavallali. Alternating optimization of decision trees, with application to learning sparse oblique trees. *Advances in neural information processing systems*, 31, 2018.

[7] Chih-Chung Chang and Chih-Jen Lin. Libsvm: A library for support vector machines. *ACM transactions on intelligent systems and technology (TIST)*, 2(3):1–27, 2011.

[8] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, page 785–794, New York, NY, USA, 2016. Association for Computing Machinery.

[9] Hyunsu Cho and Mu Li. Treelite: Toolbox for decision tree deployment, 2018.

[10] Samuel D. Curtis, Tingshan Liu, Yuxin Bai, Yuxuan Wang, Sambit Panda, Adam Li, Haoyin Xu, Eliza O'Reilly, Lisa Dobbyn, Maria Popoli, Janine Ptak, Natalie Silliman, Chris Thoburn, Jeanne Tie, Peter Gibbs, Lan T. Ho-Pham, Bich N. H. Tran, Thach S. Tran, Tuan V. Nguyen, Maximilian F. Konig, Michelle Petri, Antony Rosen, Christopher A. Mecoli, Ami A. Shah, Frits Mulder, Nick van Es, PLATO-VTE Study Group, Chetan Bettegowda, Kenneth W. Kinzler, Nickolas Papadopoulos, Joshua T. Vogelstein, Bert Vogelstein, Christopher Douville, Frederikus A. Klok, Jan Beyer-Westendorf, Luis Jara-Palomares, Walter Ageno, Mike J.L. Peters, Aurélien Delluc, Myron Best, Noori Guman, Hans-Martin Otten, Frits Mulder, and Nick van Es. Fragmentation signatures in cancer patients resemble those of patients with vascular or autoimmune diseases. *Proceedings of the National Academy of Sciences*, 122(34):e2426890122, 2025.

[11] Samuel D. Curtis, Sambit Panda, Adam Li, Haoyin Xu, Yuxin Bai, Itsuki Ogihara, Eliza O'Reilly, Yuxuan Wang, Lisa Dobbyn, Maria Popoli, Janine Ptak, Nadine Nehme, Natalie Silliman, Jeanne Tie, Peter Gibbs, Lan T. Ho-Pham, Bich N. H. Tran, Thach S. Tran, Tuan V. Nguyen, Ehsan Irajizad, Michael Goggins, Christopher L. Wolfgang, Tian-Li Wang, Ie-Ming Shih, Amanda Fader, Anne Marie Lennon, Ralph H. Hruban, Chetan Bettegowda, Lucy Gilbert, Kenneth W. Kinzler, Nickolas Papadopoulos, Bert Vogelstein, Joshua T. Vogelstein, and Christopher Douville. Minimizing and quantifying uncertainty in ai-informed decisions: Applications in medicine. *Proceedings of the National Academy of Sciences*, 122(34):e2424203122, 2025.

[12] Maanit Desai, Yirun Zhang, Ryan Holbrook, Kait O'Neil, and Maggie Demkin. Jane street real-time market data forecasting. https://www.kaggle.com/competitions/jane-street-real-time-market-data-forecasting, 2024. Kaggle.

[13] Nick Erickson, Lennart Purucker, Andrej Tschalzev, David Holzmüller, Prateek Mutalik Desai, David Salinas, and Frank Hutter. Tabarena: A living benchmark for machine learning on tabular data. *arXiv preprint arXiv:2506.16791*, 2025.

[14] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Mihir Manium, Shi Bin Hoo, Magnus Bühler, Anurag Garg, et al. Tabpfn-3: Technical report. *arXiv preprint arXiv:2605.13986*, 2026.

[15] Mathieu Guillame-Bert, Sebastian Bruch, Richard Stotz, and Jan Pfeifer. Yggdrasil decision forests: A fast and extensible decision forests library. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, pages 4068–4077, 2023.

[16] Mayuri Gupta, Ashish Mishra, Ashutosh Mishra, and Jayesh Gangrade. Accelerated gpu-based clustering, classification and regression using rapids: A comparative study. In *International Conference on Computation of Artificial Intelligence & Machine Learning*, pages 239–253. Springer, 2024.

[17] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, January 2025.

[18] Rasul Kairgeldin and Miguel Á. Carreira-Perpiñán. Bivariate Decision Trees: Smaller, Interpretable, More Accurate. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, pages 1336–1347, New York, NY, USA, August 2024. Association for Computing Machinery.

[19] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.

[20] Weihao Kong and Abhimanyu Das. Introducing TabFM: A zero-shot foundation model for tabular data. Google Research Blog, June 2026. Accessed 25 September 2026.

[21] Adam Li, Sambit Panda, Haoyin Xu, and Itsuki Ogihara. treeple: Modern decision-trees compatible with scikit-learn in python. https://github.com/neurodata/treeple, 2023. License: PolyForm-Noncommercial-1.0.0.

[22] Claudio Lucchese, Franco Maria Nardini, Salvatore Orlando, Raffaele Perego, Nicola Tonellotto, and Rossano Venturini. QuickScorer: A Fast Algorithm to Rank Documents with Additive Ensembles of Regression Trees. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '15, pages 73–82, New York, NY, USA, August 2015. Association for Computing Machinery.

[23] Meghana Madhyastha, Kunal Lillaney, James Browne, Joshua T. Vogelstein, and Randal Burns. BLOCKSET (block-aligned serialized trees): Reducing inference latency for tree ensemble deployment. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1170–1179. ACM, 2021.

[24] Jannis Maier and Lennart Purucker. Hapens: Hardware-aware post-hoc ensembling for tabular data. *arXiv preprint arXiv:2603.10582*, 2026.

[25] Andrey Malinin, Neil Band, German Chesnokov, Yarin Gal, Mark JF Gales, Alexey Noskov, Andrey Ploskonosov, Liudmila Prokhorenkova, Ivan Provilkov, Vatsal Raina, et al. Shifts: A dataset of real distributional shift across multiple large-scale tasks. *arXiv preprint arXiv:2107.07455*, 2021.

[26] Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, DB Tsai, Manish Amde, Sean Owen, et al. Mllib: Machine learning in apache spark. *Journal of Machine Learning Research*, 17(34):1–7, 2016.

[27] Sreerama K Murthy, Simon Kasif, and Steven Salzberg. A system for induction of oblique decision trees. *Journal of artificial intelligence research*, 2:1–32, 1994.

[28] Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. A tensor compiler for unified machine learning prediction serving. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 899–917, 2020.

[29] Md. Mostofa Ali Patwary, Nadathur Rajagopalan Satish, Narayanan Sundaram, Jialin Liu, Peter Sadowski, Evan Racah, Suren Byna, Craig Tull, Wahid Bhimji, Prabhat, and Pradeep Dubey. PANDA: Extreme Scale Parallel K-Nearest Neighbor on Distributed Architectures. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 494–503, May 2016. ISSN: 1530-2075.

[30] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
[31] Orestis Polychroniou and Kenneth A. Ross. A comprehensive study of main-memory partitioning and its application to large-scale comparison- and radix-sort. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, SIGMOD '14, pages 755–766, New York, NY, USA, June 2014. Association for Computing Machinery.
[32] Natalia Ponomareva, Soroush Radpour, Gilbert Hendry, Salem Haykal, Thomas Colthurst, Petr Mitrichev, and Alexander Grushetsky. Tf boosted trees: A scalable tensorflow based framework for gradient boosting. In *Joint european conference on machine learning and knowledge discovery in databases*, pages 423–427. Springer, 2017.
[33] Philipp Probst, Anne-Laure Boulesteix, and Bernd Bischl. Tunability: Importance of hyperparameters of machine learning algorithms. *Journal of Machine Learning Research*, 20(53):1–32, 2019.
[34] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. *Advances in neural information processing systems*, 31, 2018.
[35] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. Tabiclv2: A better, faster, scalable, and open tabular foundation model. *arXiv preprint arXiv:2602.11139*, 2026.
[36] Tom Rainforth and Frank Wood. Canonical correlation forests. *arXiv preprint arXiv:1507.05444*, 2015.
[37] Ivan Rubachev, Nikolay Kartashev, Yury Gorishniy, and Artem Babenko. Tabred: Analyzing pitfalls and filling the gaps in tabular deep learning benchmarks. In *International Conference on Learning Representations (ICLR)*, 2025.
[38] Benjamin Schlegel, Rainer Gemulla, and Wolfgang Lehner. K-ary search on modern processors. In *Proceedings of the Fifth International Workshop on Data Management on New Hardware*, pages 52–60, 2009.
[39] Tyler M Tomita, James Browne, Cencheng Shen, Jaewon Chung, Jesse L Patsolic, Benjamin Falk, Carey E Priebe, Jason Yim, Randal Burns, Mauro Maggioni, et al. Sparse projection oblique randomer forests. *Journal of machine learning research*, 21(104):1–39, 2020.
[40] G. V. Trunk and J. O. Coleman. Repeated hypothesis testing on a growing data set. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-4(3):343–345, 1982.
[41] Jan Wassenberg, Mark Blacher, Joachim Giesen, and Peter Sanders. Vectorized and performance-portable quicksort. *Software: Practice and Experience*, 52(12):2684–2699, 2022.
[42] Zeyi Wen, Jiashuai Shi, Bingsheng He, Qinbin Li, and Jian Chen. ThunderGBM: Fast GBDTs and random forests on GPUs. *Journal of Machine Learning Research*, 21, 2020.
[43] Daniel Whiteson. HIGGS. UCI Machine Learning Repository, 2014. DOI: https://doi.org/10.24432/C5V312.
[44] Daniel Whiteson. SUSY. UCI Machine Learning Repository, 2014. DOI: https://doi.org/10.24432/C54606.
[45] Marvin N Wright and Andreas Ziegler. ranger: A fast implementation of random forests for high dimensional data in c++ and r. *Journal of statistical software*, 77:1–17, 2017.
[46] Borui Xu, Zeyi Wen, Yao Chen, Weiguo Liu, Weng-Fai Wong, and Bingsheng He. Scalagbm: Memory efficient gbdt training for high-dimensional data on gpu. In *Proceedings of the 30th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2025.
[47] Han-Jia Ye, Si-Yang Liu, and Wei-Lun Harry Chao. A closer look at tabpfn v2: Understanding its strengths and extending its capabilities. *Advances in Neural Information Processing Systems*, 38:135605–135637, 2026.
[48] Sungduk Yu, Walter Hannah, Liran Peng, Jerry Lin, Mohamed Aziz Bhouri, Ritwik Gupta, Björn Lütjens, Justus C Will, Gunnar Behrens, Julius Busecke, et al. Climsim: A large multi-scale dataset for hybrid physics-ml climate emulation. *Advances in neural information processing systems*, 36:22070–22084, 2023.

A Appendix

A.1 `std::sort` and Highway

Dynamic histogramming was motivated by the YDF we started from, in which Exact splits sorted with `std::sort` and Random Hist. was the scalar `std::upper_bound()` implementation. Figure 10 compares the two in that setting, together with Dynamic switching between them at the per-node threshold of 1350 samples at which that pair breaks even. Sorting dominates at the top of the tree, where nodes are large; the scalar histogram dominates at the bottom, where thousands of small nodes each pay the fixed cost of setting up a histogram. Dynamic uses the histogram above the threshold and `std::sort` below it, and is the fastest of the three over the whole tree (single tree, single thread): 1.5× faster than `std::sort` (1.8× on the split search alone) and 1.1× faster than the scalar histogram, on both the dataset of Figure 10 and HIGGS.

Highway VQSort changes this picture. It trains 2.2× faster than `std::sort` and, unlike `std::sort`, it beats the scalar histogram at every depth, including the root node that holds every sample (Figure 1). Dynamic over the scalar histogram therefore no longer wins in any regime: it is 1.45× slower than Highway Exact over the whole tree. Recovering the histogram's advantage requires the vectorized histogram filling of the main text, which is why every speedup in this paper is measured against Highway Exact rather than `std::sort`.

A.2 YDF Pull Request Contributions

Due to anonymity concerns with exact Pull-request numbers and titles, we only provide a summary below.

This project has resulted in numerous improvements to the YDF library, contributed as pull requests:

- **Candidate-attribute sampling.** The eager per-node shuffle of all attributes is replaced by a lazy prefix Fisher–Yates shuffle that only draws the few positions actually tested, and is up to 75% faster on high-dimensional data.
- **Projection-evaluator caching.** We replaced an $O(F)$ object re-initialization with a cached and reused object per thread. Up to 3× faster on high-dimensional data.
- **Random-histogram split search.** Invariant code hoisted out of the candidate-threshold scan - up to 30% for sparse oblique speedup.
- **Vectorized bin generation.** `std::sort` is replaced by Highway VQSort in generating histogram bins; up to 15%.
- **Histogram split search for oblique forests.** Adds the histogram (approximate) split finder to the sparse-oblique path, which previously supported only exact splits.

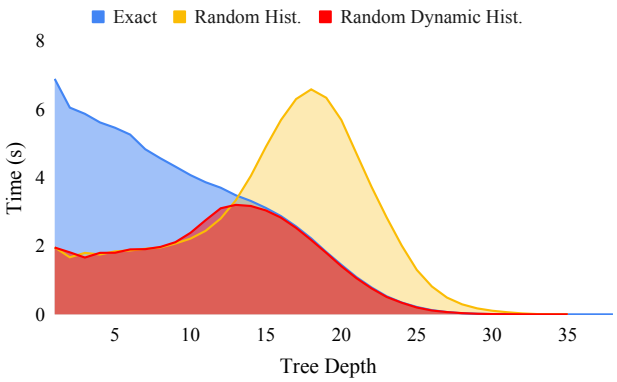


Figure 10: Training runtime by tree depth on a dataset with 1M samples 4096 features between `std::sort` and scalar Random Hist. Exact was reduced by 2-3× with the advent of Highway.

Table 5: Datasets used, by source. Categorical columns are numbered ordinally, and NaNs are column mean-imputed. Task: bin = binary classification, mul = multi-class classification, reg = regression. Multi-class classification problems was converted to 1-vs-all binary classification. Eight datasets appear in both CC18 and TabArena. † datasets were truncated to fit into memory.

Dataset	Task	Rows	Feat.
Large Natural Datasets			
HIGGS [43]	bin	11,000,000	28
SUSY [44]	bin	5,000,000	18
Epsilon [7]	bin	400,000	2,000
YouTube-8M [1]	mul*	3,888,919	1152
Shifts Weather [25]	reg*	3,129,592	127
ClimSim [48]	reg*	10,091,520†	124
Jane Street [12]	reg*	10,213,190†	82
Synthetic			
Trunk [40]	bin	-	-
OpenML CC18 [3]			
adult	bin	48,842	14
bank-marketing	bin	45,211	16
banknote-authentication	bin	1,372	4
Bioresponse	bin	3,751	1,776
blood-transfusion-service-center	bin	748	4
breast-w	bin	699	9
churn	bin	5,000	20
climate-model-simulation-crashes	bin	540	18
credit-approval	bin	690	15
credit-g	bin	1,000	20
cylinder-bands	bin	540	37
diabetes	bin	768	8
dresses-sales	bin	500	12
electricity	bin	45,312	8
ilpd	bin	583	10
Internet-Advertisements	bin	3,279	1,558
jm1	bin	10,885	21
kc1	bin	2,109	21
kc2	bin	522	21
kr-vs-kp	bin	3,196	36
madelon	bin	2,600	500
nomao	bin	34,465	118
numerai28.6	bin	96,320	21
ozone-level-8hr	bin	2,534	72
pc1	bin	1,109	21
pc3	bin	1,563	37
pc4	bin	1,458	37
PhishingWebsites	bin	11,055	30
phoneme	bin	5,404	5
qsar-biodeg	bin	1,055	41
spambase	bin	4,601	57
tic-tac-toe	bin	958	9
wdbc	bin	569	30
wilt	bin	4,839	5

Dataset	Task	Rows	Feat.
TabArena [13]			
airfoil_self_noise	reg	1,503	4
used-Fiat-500	reg	1,538	6
APSFailure	bin	76,000	170
bank-marketing	bin	45,211	5
Bank_Customer_Churn	bin	10,000	6
Bioresponse	bin	3,751	1,776
blood-transfusion	bin	748	4
churn	bin	5,000	15
coil2000_insurance_policies	bin	9,822	80
concrete_compressive	reg	1,030	8
credit-g	bin	1,000	7
credit_card_default	bin	30,000	20
customer_satisf._airline	bin	129,880	5
diabetes	bin	768	8
Diabetes130US	bin	71,518	8
diamonds	reg	53,940	6
E-CommereShippingData	bin	10,999	6
Fitness_Club	bin	1,500	3
Food_Delivery_Time	reg	45,451	6
GiveMeSomeCredit	bin	150,000	10
hazelnut-spread-contam.	bin	2,400	30
healthcare_insurance	reg	1,338	3
heloc	bin	10,459	23
houses	reg	20,640	8
HR_Analytics_Job_Change	bin	19,158	2
in_vehicle_coupon	bin	12,684	2
Is-this-a-good-customer	bin	1,723	5
jm1	bin	10,885	21
Marketing_Campaign	bin	2,240	16
miami_housing	reg	13,776	14
online_shoppers_intention	bin	12,330	10
physiochemical_protein	reg	45,730	9
polish_bankruptcy	bin	5,910	64
qsar-biodeg	bin	1,054	36
QSAR-TID-11	reg	5,742	1,024
QSAR_fish_toxicity	reg	907	6
seismic-bumps	bin	2,584	11
superconductivity	reg	21,263	81
taiwanese_bankruptcy	bin	6,819	94
wine_quality	reg	6,497	11
TabReD [37]			
cooking-time	reg	319,986	186
ecom-offers	bin	160,057	113
homesite-insurance	bin	260,753	253
weather	reg	423,795	100

- **Equal-width binning.** Samples are assigned to equal-width bins with $O(1)$ arithmetic instead of a binary search over the bin boundaries.
- **Projection sampling.** The per-feature uniform draws that build each sparse projection are replaced by one Binomial draw for the number of non-zeros plus Floyd's sampler for their positions, reducing the cost from linear in the number of features to linear in the number of non-zeros (proof below).

Floyd's projection sampling. Originally, random sampling to build the projection matrix was a computational bottleneck for SO-YDF. This component comprised 80% of the runtime, with the math-heavy parts of the code taking <20%. The previous design assumed small d and was inefficient for wide data.

Existing code calls Unif(0,1) over the projection matrix to create a mask, then populates this mask with weights. However, this requires $\Theta(np)$ calls to `Unif(·)`, or p calls per each projection, across n projections. We show this can be substituted with a single call to Binomial($np, n\frac{k}{p}$), cutting runtime by 33% for our settings, proportional to $\Theta(np)$.

Using a variant of an algorithm from Floyd [2], we form the projection matrix as follows:

```
projection_density = k/p;
for projection in projections:  // O(n)
    for feature in features:    // O(p)
        if (Unif01(.) < projection_density) {...}
```

Dataset	Baselines			Our Methods			
	Exact	Random Hist.	Random Hist.	Vec. Random Hist.	Vec. Random Hist.	Vec. Dynamic Hist.	Vec. Dynamic Hist.
	HWY	scalar, 64 bins	scalar, 256 bins	AVX2, 64 bins	AVX-512, 256 bins	AVX2, 64 bins & HWY	AVX-512, 256 bins & HWY
Depth 6							
HIGGS 10.5M×28	79.6	59.3	70.1	43.3 (1.8×)	43.9 (1.8×)	44.7 (1.8×)	44.3 (1.8×)
SUSY 4.5M×18	27.0	21.3	25.4	14.9 (1.8×)	14.9 (1.8×)	14.9 (1.8×)	15.1 (1.8×)
Epsilon 400k×2000	12.0	13.6	17.3	7.4 (1.6×)	7.9 (1.5×)	7.4 (1.6×)	7.5 (1.6×)
GiveMeSomeCredit 150k×10	0.3	0.4	0.4	0.3 (1.1×)	0.3 (1.1×)	0.3 (1.0×)	0.3 (1.2×)
YouTube-8M 3.9M×1152	137	105	127	70.7 (1.9×)	74.6 (1.8×)	70.9 (1.9×)	74.1 (1.8×)
Shifts weather 3.1M×127	34.3	27.9	33.2	20.2 (1.7×)	20.0 (1.7×)	20.0 (1.7×)	19.7 (1.7×)
ClimSim 10.1M×124	146	98.2	117	72.7 (2.0×)	72.9 (2.0×)	73.2 (2.0×)	71.9 (2.0×)
Jane Street 10.2M×82	121	74.5	86.7	60.2 (2.0×)	60.7 (2.0×)	60.1 (2.0×)	60.1 (2.0×)
Trunk 1M×32	5.2	5.2	6.3	3.2 (1.6×)	3.3 (1.6×)	3.3 (1.6×)	3.2 (1.6×)
Trunk 1M×512	18.8	20.0	22.8	11.2 (1.7×)	11.1 (1.7×)	10.9 (1.7×)	11.0 (1.7×)
Trunk 1M×2048	35.1	36.1	45.1	21.2 (1.7×)	21.8 (1.6×)	20.9 (1.7×)	21.6 (1.6×)
Trunk 15k×400k	5.0	7.3	9.7	3.5 (1.4×)	3.9 (1.3×)	3.5 (1.4×)	3.5 (1.4×)
Trunk 15k×1.6M	8.7	12.0	16.0	6.1 (1.4×)	6.7 (1.3×)	6.0 (1.5×)	6.3 (1.4×)
Trunk 150k×4096	5.6	7.2	9.3	3.6 (1.6×)	3.8 (1.5×)	3.6 (1.5×)	3.9 (1.4×)
Trunk 150k×40k	17.2	22.0	28.8	11.2 (1.5×)	11.8 (1.5×)	11.0 (1.6×)	11.4 (1.5×)
Trunk 150k×160k	32.9	43.4	56.2	21.2 (1.6×)	21.7 (1.5×)	21.2 (1.6×)	22.4 (1.5×)
Trunk 150k×400k	51.8	67.5	88.1	32.5 (1.6×)	34.1 (1.5×)	32.4 (1.6×)	33.7 (1.5×)
Trunk 1.5M×4096	78.1	75.6	94.7	45.8 (1.7×)	47.3 (1.7×)	45.8 (1.7×)	47.4 (1.6×)
Trunk 1.5M×16384	155	150	187	89.8 (1.7×)	90.2 (1.7×)	90.9 (1.7×)	91.6 (1.7×)
Trunk 1.5M×40k	243	234	293	145 (1.7×)	145 (1.7×)	144 (1.7×)	140 (1.7×)
Trunk 4.5M×4096	296	236	290	158 (1.9×)	159 (1.9×)	158 (1.9×)	163 (1.8×)
Depth 10							
HIGGS 10.5M×28	141	116	138	89.7 (1.6×)	91.5 (1.5×)	90.1 (1.6×)	91.2 (1.5×)
SUSY 4.5M×18	46.1	40.8	49.1	28.5 (1.6×)	28.6 (1.6×)	28.4 (1.6×)	28.5 (1.6×)
Epsilon 400k×2000	23.7	28.2	35.6	16.7 (1.4×)	17.7 (1.3×)	16.8 (1.4×)	17.1 (1.4×)
GiveMeSomeCredit 150k×10	0.5	0.6	0.8	0.5 (1.0×)	0.5 (1.0×)	0.4 (1.1×)	0.5 (1.1×)
YouTube-8M 3.9M×1152	235	211	251	151 (1.5×)	153 (1.5×)	151 (1.6×)	153 (1.5×)
Shifts weather 3.1M×127	58.1	53.4	62.5	39.1 (1.5×)	39.0 (1.5×)	39.4 (1.5×)	38.6 (1.5×)
ClimSim 10.1M×124	228	175	206	129 (1.8×)	127 (1.8×)	128 (1.8×)	127 (1.8×)
Jane Street 10.2M×82	202	144	167	119 (1.7×)	120 (1.7×)	119 (1.7×)	119 (1.7×)
Trunk 1M×32	8.7	9.6	11.9	5.7 (1.5×)	5.9 (1.5×)	5.8 (1.5×)	5.8 (1.5×)
Trunk 1M×512	33.6	37.6	46.2	23.1 (1.5×)	23.7 (1.4×)	23.1 (1.5×)	23.1 (1.5×)
Trunk 1M×2048	66.1	74.0	91.1	45.7 (1.4×)	46.6 (1.4×)	45.3 (1.5×)	46.2 (1.4×)
Trunk 15k×400k	10.6	15.9	21.8	9.1 (1.2×)	11.2 (0.9×)	8.3 (1.3×)	8.8 (1.2×)
Trunk 15k×1.6M	18.2	25.9	35.7	15.1 (1.2×)	18.9 (1.0×)	13.8 (1.3×)	14.8 (1.2×)
Trunk 150k×4096	11.8	15.2	19.6	8.7 (1.4×)	9.3 (1.3×)	8.7 (1.4×)	8.9 (1.3×)
Trunk 150k×40k	36.6	46.5	59.6	26.8 (1.4×)	28.4 (1.3×)	26.4 (1.4×)	27.0 (1.4×)
Trunk 150k×160k	69.5	89.6	116	49.7 (1.4×)	52.8 (1.3×)	49.8 (1.4×)	50.3 (1.4×)
Trunk 150k×400k	107	137	180	74.1 (1.4×)	82.5 (1.3×)	73.8 (1.4×)	76.7 (1.4×)
Trunk 1.5M×4096	143	155	192	99.3 (1.4×)	98.3 (1.5×)	98.0 (1.5×)	97.9 (1.5×)
Trunk 1.5M×16384	282	304	378	190 (1.5×)	194 (1.5×)	190 (1.5×)	193 (1.5×)
Trunk 1.5M×40k	440	473	587	304 (1.4×)	309 (1.4×)	304 (1.4×)	300 (1.5×)
Trunk 4.5M×4096	496	472	576	321 (1.5×)	324 (1.5×)	322 (1.5×)	328 (1.5×)
Depth 16							
HIGGS 10.5M×28	211	204	243	150 (1.4×)	154 (1.4×)	150 (1.4×)	151 (1.4×)
SUSY 4.5M×18	68.4	69.9	86.4	46.4 (1.5×)	48.1 (1.4×)	46.6 (1.5×)	46.4 (1.5×)
Epsilon 400k×2000	41.8	55.3	78.8	35.2 (1.2×)	46.3 (0.9×)	32.5 (1.3×)	34.0 (1.2×)
GiveMeSomeCredit 150k×10	0.8	1.1	1.4	0.8 (1.0×)	1.0 (0.8×)	0.8 (1.1×)	0.8 (1.1×)
YouTube-8M 3.9M×1152	365	379	464	257 (1.4×)	266 (1.4×)	257 (1.4×)	260 (1.4×)
Shifts weather 3.1M×127	82.5	84.8	103	60.6 (1.4×)	62.9 (1.3×)	60.5 (1.4×)	60.1 (1.4×)
ClimSim 10.1M×124	271	232	276	168 (1.6×)	167 (1.6×)	167 (1.6×)	163 (1.7×)
Jane Street 10.2M×82	301	258	304	205 (1.5×)	206 (1.5×)	205 (1.5×)	204 (1.5×)
Trunk 1M×32	12.9	16.5	22.3	10.0 (1.3×)	12.2 (1.1×)	9.5 (1.4×)	9.6 (1.3×)
Trunk 1M×512	56.1	70.3	93.8	44.7 (1.3×)	53.5 (1.0×)	42.7 (1.3×)	43.2 (1.3×)
Trunk 1M×2048	114	141	187	89.9 (1.3×)	106 (1.1×)	86.5 (1.3×)	87.1 (1.3×)
Trunk 15k×400k	20.2	31.5	48.7	21.3 (0.9×)	31.1 (0.6×)	17.2 (1.2×)	18.1 (1.1×)
Trunk 15k×1.6M	36.4	52.3	79.1	36.2 (1.0×)	52.5 (0.7×)	30.9 (1.2×)	32.2 (1.1×)
Trunk 150k×4096	21.5	31.3	46.8	20.4 (1.1×)	29.0 (0.7×)	17.7 (1.2×)	17.9 (1.2×)
Trunk 150k×40k	68.5	95.7	135	60.6 (1.1×)	81.7 (0.8×)	54.8 (1.2×)	56.3 (1.2×)
Trunk 150k×160k	135	183	257	115 (1.2×)	150 (0.9×)	107 (1.3×)	112 (1.2×)
Trunk 150k×400k	210	276	387	171 (1.2×)	216 (1.0×)	164 (1.3×)	168 (1.3×)
Trunk 1.5M×4096	247	295	379	190 (1.3×)	212 (1.2×)	186 (1.3×)	187 (1.3×)
Trunk 1.5M×16384	500	596	758	384 (1.3×)	425 (1.2×)	377 (1.3×)	380 (1.3×)
Trunk 1.5M×40k	779	918	1153	589 (1.3×)	641 (1.2×)	581 (1.3×)	585 (1.3×)
Trunk 4.5M×4096	826	899	1115	597 (1.4×)	625 (1.3×)	595 (1.4×)	601 (1.4×)

Table 6: (Part 1 of 2: depth 6, depth 10, depth 16.) End-to-end SPO-RF training time (s) by tree depth.

Dataset	Baselines			Our Methods			
	Exact	Random Hist.	Random Hist.	Vec. Random Hist.	Vec. Random Hist.	Vec. Dynamic Hist.	Vec. Dynamic Hist.
	HWY	scalar, 64 bins	scalar, 256 bins	AVX2, 64 bins	AVX-512, 256 bins	AVX2, 64 bins & HWY	AVX-512, 256 bins & HWY
Depth 20							
HIGGS 10.5M×28	249	254	312	184 (1.3×)	194 (1.3×)	183 (1.4×)	185 (1.3×)
SUSY 4.5M×18	80.6	86.3	110	57.3 (1.4×)	61.8 (1.3×)	55.9 (1.4×)	56.7 (1.4×)
Epsilon 400k×2000	51.0	76.2	117	49.9 (1.0×)	76.3 (0.7×)	42.1 (1.2×)	43.9 (1.2×)
GiveMeSomeCredit 150k×10	1.0	1.4	2.0	1.0 (0.9×)	1.4 (0.7×)	1.0 (1.0×)	0.9 (1.0×)
YouTube-8M 3.9M×1152	438	479	615	326 (1.3×)	365 (1.2×)	320 (1.4×)	324 (1.4×)
Shifts weather 3.1M×127	92.2	97.1	123	71.0 (1.3×)	78.2 (1.2×)	68.9 (1.3×)	69.5 (1.3×)
ClimSim 10.1M×124	298	256	318	190 (1.6×)	196 (1.5×)	189 (1.6×)	187 (1.6×)
Jane Street 10.2M×82	374	338	413	267 (1.4×)	278 (1.3×)	264 (1.4×)	267 (1.4×)
Trunk 1M×32	13.6	19.0	27.5	11.5 (1.2×)	15.9 (0.9×)	10.1 (1.4×)	10.3 (1.3×)
Trunk 1M×512	64.1	90.4	133	58.9 (1.1×)	84.3 (0.8×)	51.4 (1.2×)	53.3 (1.2×)
Trunk 1M×2048	133	186	275	121 (1.1×)	173 (0.8×)	106 (1.2×)	110 (1.2×)
Trunk 15k×400k	23.8	38.0	59.0	25.6 (0.9×)	39.6 (0.6×)	20.8 (1.1×)	21.4 (1.1×)
Trunk 15k×1.6M	43.2	65.2	96.1	46.2 (0.9×)	65.4 (0.7×)	38.2 (1.1×)	40.2 (1.1×)
Trunk 150k×4096	23.7	38.3	61.5	25.0 (0.9×)	40.8 (0.6×)	20.2 (1.2×)	21.1 (1.1×)
Trunk 150k×40k	81.1	123	187	80.6 (1.0×)	121 (0.7×)	68.6 (1.2×)	71.1 (1.1×)
Trunk 150k×160k	170	245	351	161 (1.1×)	222 (0.8×)	141 (1.2×)	148 (1.1×)
Trunk 150k×400k	267	374	534	241 (1.1×)	327 (0.8×)	221 (1.2×)	230 (1.2×)
Trunk 1.5M×4096	296	393	559	258 (1.1×)	353 (0.8×)	232 (1.3×)	241 (1.2×)
Trunk 1.5M×16384	636	819	1122	548 (1.2×)	705 (0.9×)	505 (1.3×)	525 (1.2×)
Trunk 1.5M×40k	1001	1263	1682	841 (1.2×)	1029 (1.0×)	791 (1.3×)	816 (1.2×)
Trunk 4.5M×4096	1036	1227	1634	837 (1.2×)	1016 (1.0×)	795 (1.3×)	821 (1.3×)
Depth 24							
HIGGS 10.5M×28	280	325	434	225 (1.2×)	271 (1.0×)	215 (1.3×)	215 (1.3×)
SUSY 4.5M×18	91.0	109	148	71.3 (1.3×)	87.2 (1.0×)	66.2 (1.4×)	65.9 (1.4×)
Epsilon 400k×2000	58.0	90.5	147	62.7 (0.9×)	103 (0.6×)	49.1 (1.2×)	49.3 (1.2×)
GiveMeSomeCredit 150k×10	1.2	1.7	2.4	1.3 (0.9×)	1.9 (0.6×)	1.1 (1.0×)	1.1 (1.0×)
YouTube-8M 3.9M×1152	477	567	768	380 (1.3×)	474 (1.0×)	356 (1.3×)	364 (1.3×)
Shifts weather 3.1M×127	92.4	105	138	74.6 (1.2×)	89.0 (1.0×)	70.5 (1.3×)	69.5 (1.3×)
ClimSim 10.1M×124	299	280	354	200 (1.5×)	220 (1.4×)	194 (1.5×)	192 (1.6×)
Jane Street 10.2M×82	421	422	549	319 (1.3×)	364 (1.2×)	309 (1.4×)	311 (1.4×)
Trunk 1M×32	14.2	19.8	28.9	12.5 (1.1×)	17.7 (0.8×)	10.7 (1.3×)	10.7 (1.3×)
Trunk 1M×512	68.5	98.4	150	66.5 (1.0×)	100 (0.7×)	54.9 (1.2×)	55.6 (1.2×)
Trunk 1M×2048	146	213	328	146 (1.0×)	223 (0.7×)	120 (1.2×)	120 (1.2×)
Trunk 15k×400k	26.2	41.9	66.4	28.7 (0.9×)	45.8 (0.6×)	22.9 (1.1×)	23.0 (1.1×)
Trunk 15k×1.6M	49.7	75.0	111	54.9 (0.9×)	79.3 (0.6×)	45.8 (1.1×)	45.3 (1.1×)
Trunk 150k×4096	25.1	41.6	67.1	27.9 (0.9×)	46.4 (0.5×)	21.6 (1.2×)	22.0 (1.1×)
Trunk 150k×40k	90.9	141	221	95.5 (1.0×)	150 (0.6×)	77.2 (1.2×)	78.2 (1.2×)
Trunk 150k×160k	202	297	447	203 (1.0×)	300 (0.7×)	173 (1.2×)	175 (1.2×)
Trunk 150k×400k	330	472	695	321 (1.0×)	458 (0.7×)	277 (1.2×)	282 (1.2×)
Trunk 1.5M×4096	338	477	714	326 (1.0×)	483 (0.7×)	273 (1.2×)	274 (1.2×)
Trunk 1.5M×16384	758	1027	1494	718 (1.1×)	1018 (0.7×)	623 (1.2×)	626 (1.2×)
Trunk 1.5M×40k	1221	1628	2320	1134 (1.1×)	1561 (0.8×)	1008 (1.2×)	1017 (1.2×)
Trunk 4.5M×4096	1233	1571	2247	1102 (1.1×)	1511 (0.8×)	977 (1.3×)	980 (1.3×)
Full depth (purity)							
HIGGS 10.5M×28	364	524	791	385 (0.9×)	569 (0.6×)	298 (1.2×)	291 (1.2×)
SUSY 4.5M×18	118	166	259	116 (1.0×)	180 (0.7×)	90.9 (1.3×)	90.9 (1.3×)
Epsilon 400k×2000	59.9	96.1	156	66.3 (0.9×)	111 (0.5×)	50.8 (1.2×)	50.7 (1.2×)
GiveMeSomeCredit 150k×10	1.2	1.9	2.8	1.5 (0.8×)	2.2 (0.6×)	1.2 (1.0×)	1.2 (1.1×)
YouTube-8M 3.9M×1152	504	613	852	423 (1.2×)	549 (0.9×)	379 (1.3×)	385 (1.3×)
Shifts weather 3.1M×127	93.9	109	146	77.8 (1.2×)	95.9 (1.0×)	72.0 (1.3×)	70.8 (1.3×)
ClimSim 10.1M×124	308	298	394	215 (1.4×)	253 (1.2×)	203 (1.5×)	198 (1.6×)
Jane Street 10.2M×82	593	792	1246	604 (1.0×)	933 (0.6×)	479 (1.2×)	480 (1.2×)
Trunk 1M×32	14.7	19.7	28.9	12.5 (1.2×)	17.7 (0.8×)	11.0 (1.3×)	10.6 (1.4×)
Trunk 1M×512	69.8	99.2	150	67.1 (1.0×)	100 (0.7×)	56.9 (1.2×)	55.7 (1.3×)
Trunk 1M×2048	151	218	333	148 (1.0×)	227 (0.7×)	125 (1.2×)	121 (1.2×)
Trunk 15k×400k	26.0	43.2	69.8	29.7 (0.9×)	47.7 (0.5×)	22.9 (1.1×)	23.9 (1.1×)
Trunk 15k×1.6M	50.4	78.9	119	57.1 (0.9×)	84.3 (0.6×)	45.5 (1.1×)	47.1 (1.1×)
Trunk 150k×4096	24.7	41.5	67.3	27.8 (0.9×)	46.4 (0.5×)	21.0 (1.2×)	21.9 (1.1×)
Trunk 150k×40k	90.6	148	230	101 (0.9×)	158 (0.6×)	77.2 (1.2×)	80.6 (1.1×)
Trunk 150k×160k	211	332	503	229 (0.9×)	340 (0.6×)	183 (1.2×)	189 (1.1×)
Trunk 150k×400k	363	551	830	380 (1.0×)	555 (0.7×)	312 (1.2×)	322 (1.1×)
Trunk 1.5M×4096	332	490	745	338 (1.0×)	513 (0.6×)	270 (1.2×)	280 (1.2×)
Trunk 1.5M×16384	766	1107	1654	787 (1.0×)	1148 (0.7×)	642 (1.2×)	663 (1.2×)
Trunk 1.5M×40k	1304	1845	2738	1314 (1.0×)	1896 (0.7×)	1096 (1.2×)	1126 (1.2×)
Trunk 4.5M×4096	1242	1683	2467	1194 (1.0×)	1701 (0.7×)	996 (1.2×)	1023 (1.2×)

Table 6: (Part 2 of 2: depth 20, depth 24, full depth (purity).) End-to-end SPO-RF training time (s) by tree depth.

Dataset	Baselines			Our Methods	
	Exact	Random Hist.	Random Hist.	Vec. Random Hist.	Vec. Random Hist.
	HWY	scalar, 64 bins	scalar, 256 bins	AVX2, 64 bins	AVX-512, 256 bins
Depth 6					
HIGGS 10.5M×28	1619	2519	3341	1243 (1.3×)	1220 (1.3×)
SUSY 4.5M×18	638	897	1210	456 (1.4×)	437 (1.5×)
Epsilon 400k×2000	96.5	159	216	74.8 (1.3×)	78.9 (1.2×)
GiveMeSomeCredit 150k×10	12.9	17.1	20.1	11.7 (1.1×)	12.1 (1.1×)
YouTube-8M 3.9M×1152	1021	1458	1964	666 (1.5×)	674 (1.5×)
Shifts weather 3.1M×127	491	717	958	353 (1.4×)	354 (1.4×)
ClimSim 10.1M×124	1741	2258	3201	1118 (1.6×)	1113 (1.6×)
Jane Street 10.2M×82	2286	2651	3578	1429 (1.6×)	1404 (1.6×)
Trunk 1M×32	146	248	335	105 (1.4×)	106 (1.4×)
Trunk 1M×512	207	345	467	157 (1.3×)	160 (1.3×)
Trunk 1M×2048	249	424	576	191 (1.3×)	201 (1.2×)
Trunk 15k×400k	57.8	65.2	70.5	55.8 (1.0×)	55.8 (1.0×)
Trunk 15k×1.6M	253	271	282	250 (1.0×)	251 (1.0×)
Trunk 150k×4096	41.0	69.7	94.7	34.2 (1.2×)	35.4 (1.2×)
Trunk 150k×40k	55.8	87.4	113	47.7 (1.2×)	49.3 (1.1×)
Trunk 150k×160k	84.3	112	143	62.6 (1.3×)	66.6 (1.3×)
Trunk 150k×400k	126	176	220	105 (1.2×)	114 (1.1×)
Trunk 1.5M×4096	393	643	869	297 (1.3×)	324 (1.2×)
Trunk 1.5M×16384	432	672	897	354 (1.2×)	377 (1.1×)
Trunk 1.5M×40k	464	693	914	352 (1.3×)	384 (1.2×)
Trunk 4.5M×4096	1278	1927	2579	902 (1.4×)	936 (1.4×)

Table 7: End-to-end SPO-GBT training time (s) at depth 6.

Dataset	Depth 6		Depth 10		Depth 16		Depth 24		Full depth (purity)	
	Exact (`std::sort`)	Exact (Highway [41])	Exact (`std::sort`)	Exact (Highway [41])	Exact (`std::sort`)	Exact (Highway [41])	Exact (`std::sort`)	Exact (Highway [41])	Exact (`std::sort`)	Exact (Highway [41])
HIGGS 10.5M×28	146	79.7 (1.8×)	257	142 (1.8×)	408	209 (2.0×)	560	277 (2.0×)	694	352 (2.0×)
SUSY 4.5M×18	57.0	27.6 (2.1×)	98.5	46.3 (2.1×)	151	68.1 (2.2×)	200	89.7 (2.2×)	247	116 (2.1×)
Epsilon 400k×2000	36.0	11.9 (3.0×)	61.9	23.5 (2.6×)	94.3	41.4 (2.3×)	117	56.5 (2.1×)	119	58.0 (2.0×)
GiveMeSomeCredit 150k×10	0.8	0.3 (2.6×)	1.3	0.5 (2.6×)	2.0	0.8 (2.4×)	2.4	1.1 (2.2×)	2.6	1.2 (2.2×)
Trunk 1M×32	13.4	5.2 (2.6×)	22.3	8.6 (2.6×)	32.1	12.9 (2.5×)	34.1	14.0 (2.4×)	34.1	14.0 (2.4×)
Trunk 1M×512	49.2	17.9 (2.7×)	84.3	33.0 (2.6×)	127	54.8 (2.3×)	145	66.8 (2.2×)	145	66.7 (2.2×)
Trunk 1M×2048	97.0	34.9 (2.8×)	167	65.0 (2.6×)	254	110 (2.3×)	303	143 (2.1×)	305	144 (2.1×)
Trunk 15k×400k	15.1	5.0 (3.0×)	26.3	10.5 (2.5×)	41.6	19.9 (2.1×)	51.0	25.6 (2.0×)	52.8	26.5 (2.0×)
Trunk 15k×1.6M	24.6	8.6 (2.9×)	42.8	18.0 (2.4×)	68.6	35.9 (1.9×)	88.0	48.5 (1.8×)	93.2	51.5 (1.8×)
Trunk 150k×4096	17.7	5.5 (3.2×)	30.5	11.6 (2.6×)	46.1	20.7 (2.2×)	51.5	24.6 (2.1×)	51.8	24.7 (2.1×)
Trunk 150k×40k	55.3	17.0 (3.2×)	95.6	35.8 (2.7×)	150	66.6 (2.2×)	186	89.3 (2.1×)	191	91.3 (2.1×)
Trunk 150k×160k	112	32.6 (3.4×)	195	68.6 (2.8×)	310	133 (2.3×)	416	199 (2.1×)	438	214 (2.0×)
Trunk 150k×400k	177	50.9 (3.5×)	310	105 (2.9×)	498	206 (2.4×)	687	322 (2.1×)	761	368 (2.1×)
Trunk 1.5M×4096	208	78.3 (2.7×)	358	141 (2.5×)	549	240 (2.3×)	683	330 (2.1×)	692	334 (2.1×)
Trunk 1.5M×16384	414	155 (2.7×)	716	279 (2.6×)	1118	489 (2.3×)	1487	739 (2.0×)	1543	772 (2.0×)
Trunk 1.5M×40k	649	243 (2.7×)	1127	439 (2.6×)	1764	762 (2.3×)	2450	1199 (2.0×)	2617	1313 (2.0×)
Trunk 4.5M×4096	680	305 (2.2×)	1168	502 (2.3×)	1795	816 (2.2×)	2400	1207 (2.0×)	2466	1253 (2.0×)

Table 8: End-to-end SPO-RF training time (s) of the Exact split finder with `std::sort` vs. Highway VQSort, by tree depth. Speedup of HWY over `std::sort` in parentheses.

Table 9: RF (full-depth, top) and GBT (depth 6, bottom) test ROC AUC on various datasets. Cells are the mean ± std over 5 CV folds. Bold: best per row. ecom-offers, homecredit-default, homesite-insurance are single chronological holdouts. XGBoost, LightGBM, CatBoost used with default parameters.

Dataset	SPO-RF Exact	SPO-RF Random Hist. (64 bins)	SPO-RF Random Hist. (256 bins)	SPO-RF AVX-2 R. Hist. (64 bins, ours)	SPO-RF AVX-512 R. Hist. (256 bins, ours)	SPO-RF Dyn AVX-2 R. Hist. (64 bins, ours)	SPO-RF Dyn AVX-512 R. Hist. (256 bins, ours)	AA-RF Exact	XGBoost RF mode†	LightGBM RF mode (255 bins)	CatBoost RF mode†
APSFailure	.9893±.0056	.9885±.0054	.9886±.0053	.9885±.0054	.9886±.0053	.9890±.0050	.9889±.0054	.9887±.0052	—	**.9900±.0047**	—
Amazon_employee_access	.8586±.0094	.8601±.0109	.8587±.0090	.8601±.0109	.8587±.0090	.8589±.0093	.8573±.0092	.8549±.0133	—	**.8614±.0110**	—
Bank_Customer_Churn	.8559±.0049	**.8568±.0056**	**.8568±.0053**	**.8568±.0056**	**.8568±.0053**	.8552±.0053	.8551±.0044	.8527±.0040	—	.8548±.0069	—
Bioresponse	.8728±.0169	.8739±.0151	.8732±.0153	.8739±.0151	.8732±.0153	.8736±.0158	**.8744±.0149**	**.8744±.0143**	—	.8741±.0182	—
Diabetes130US	**.6447±.0058**	.6393±.0038	.6412±.0055	.6393±.0038	.6412±.0055	.6427±.0049	.6422±.0037	.6383±.0019	—	.6310±.0047	—
E-CommereShippingData	.7378±.0077	.7387±.0076	**.7396±.0081**	.7387±.0076	**.7396±.0081**	.7387±.0083	.7378±.0089	.7386±.0080	—	.7382±.0089	—
Fitness_Club	.7768±.0111	.7768±.0106	.7794±.0120	.7768±.0106	.7794±.0120	.7766±.0122	.7758±.0098	.7716±.0096	—	**.7796±.0083**	—
GiveMeSomeCredit	.8489±.0034	.8458±.0025	.8460±.0032	.8458±.0025	.8460±.0032	.8453±.0027	.8468±.0033	.8405±.0021	—	**.8523±.0026**	—
HR_Analytics_Job_Change_of_Data_Scientists	.7918±.0084	.7927±.0080	.7929±.0092	.7927±.0080	.7929±.0092	.7923±.0096	.7913±.0079	.7873±.0078	—	**.7946±.0091**	—
Is-this-a-good-customer	.7219±.0353	.7224±.0350	.7222±.0431	.7224±.0350	.7222±.0431	.7211±.0341	.7253±.0280	**.7324±.0287**	—	.7185±.0644	—
Marketing_Campaign	.8945±.0244	.8962±.0223	.8956±.0241	.8962±.0223	.8956±.0241	.8975±.0235	.8938±.0250	**.8998±.0244**	—	.8955±.0301	—
NATICUSdroid	.9826±.0038	.9828±.0038	.9828±.0036	.9828±.0038	.9828±.0036	.9828±.0033	.9825±.0036	.9763±.0034	—	**.9839±.0038**	—
bank-marketing	.7396±.0055	.7396±.0044	**.7397±.0061**	.7396±.0044	**.7397±.0061**	.7388±.0063	.7394±.0050	.7223±.0075	—	.7304±.0048	—
blood-transfusion-service-center	.7075±.0478	.7110±.0540	**.7150±.0496**	.7110±.0540	**.7150±.0496**	.7062±.0473	.7041±.0478	.6960±.0348	—	.7091±.0405	—
churn	.9171±.0254	.9160±.0260	**.9175±.0235**	.9160±.0260	**.9175±.0235**	.9164±.0261	.9144±.0275	.9174±.0262	—	.9124±.0251	—
coil2000_insurance_policies	.7084±.0129	.7024±.0120	.7095±.0136	.7024±.0120	.7095±.0136	**.7127±.0123**	.7044±.0141	.7101±.0157	—	.7039±.0155	—
credit-g	**.7836±.0290**	.7764±.0331	.7824±.0361	.7764±.0331	.7824±.0361	**.7836±.0419**	.7804±.0328	.7810±.0380	—	.7684±.0372	—
credit_card_clients_default	.7699±.0081	.7706±.0081	.7703±.0077	.7706±.0081	.7703±.0077	.7700±.0075	**.7719±.0077**	.7676±.0078	—	.7678±.0082	—
customer_satisfaction_in_airline	.9924±.0003	.9926±.0003	.9925±.0003	.9926±.0003	.9925±.0003	.9926±.0003	.9925±.0003	**.9935±.0003**	—	.9931±.0003	—
diabetes	.8285±.0129	.8291±.0176	.8295±.0196	.8291±.0176	.8295±.0196	**.8298±.0223**	.8295±.0180	.8267±.0206	—	.8223±.0221	—
ecom-offers	.6164	.6185	.6199	.6185	.6199	**.6228**	.6177	.6075	—	.6167	—
hazelnut-spread-contaminant-detection	.9628±.0090	**.9641±.0094**	.9628±.0096	**.9641±.0094**	.9628±.0096	.9633±.0096	.9636±.0087	.9629±.0106	—	.9585±.0097	—
heloc	.7921±.0112	.7924±.0115	.7927±.0117	.7924±.0115	.7927±.0117	.7921±.0132	.7921±.0107	**.7932±.0126**	—	.7924±.0118	—
homecredit-default	.8072	**.8116**	.8091	**.8116**	.8091	.8044	.8110	.8034	—	.8011	—
homesite-insurance	.9416	.9428	.9427	.9428	.9427	.9432	.9419	**.9472**	—	.9414	—
in_vehicle_coupon_recommendation	.8198±.0049	.8209±.0041	.8213±.0050	.8209±.0041	.8213±.0050	.8208±.0033	.8209±.0047	**.8248±.0053**	—	.8198±.0054	—
jm1	.7625±.0106	.7616±.0092	.7622±.0078	.7616±.0092	.7622±.0078	**.7633±.0103**	.7614±.0104	.7548±.0092	—	.7609±.0112	—
kddcup09_appetency	**.7842±.0151**	.7795±.0126	.7817±.0100	.7795±.0126	.7817±.0100	.7825±.0173	.7800±.0195	.7794±.0127	—	.7701±.0286	—
online_shoppers_intention	.9257±.0071	.9267±.0066	.9271±.0065	.9267±.0066	.9271±.0065	.9260±.0064	.9271±.0060	**.9278±.0048**	—	.9160±.0077	—
polish_companies_bankruptcy	**.9385±.0099**	.9251±.0107	.9284±.0109	.9251±.0107	.9284±.0109	.9243±.0099	.9302±.0087	.9381±.0095	—	.9289±.0086	—
qsar-biodeg	.9347±.0128	.9363±.0134	.9352±.0142	.9363±.0134	.9352±.0142	**.9372±.0141**	.9350±.0121	.9329±.0157	—	.9305±.0142	—
seismic-bumps	.7427±.0630	.7417±.0719	**.7491±.0635**	.7417±.0719	**.7491±.0635**	.7400±.0616	.7435±.0677	.7400±.0555	—	.7427±.0702	—
taiwanese_bankruptcy_prediction	.9434±.0215	.9393±.0194	.9384±.0213	.9393±.0194	.9384±.0213	.9427±.0199	.9420±.0216	**.9437±.0207**	—	.9359±.0266	—

Dataset	SPO-GBT Exact	SPO-GBT Random Hist. (64 bins)	SPO-GBT Random Hist. (256 bins)	SPO-GBT AVX-2 R. Hist. (64 bins, ours)	SPO-GBT AVX-512 R. Hist. (256 bins, ours)	SPO-GBT Dyn AVX-2 R. Hist. (64 bins, ours)*	SPO-GBT Dyn AVX-512 R. Hist. (256 bins, ours)*	AA-GBT Exact	XGBoost (256 bins)	LightGBM (255 bins)	CatBoost (254 bins)
APSFailure	.9917±.0033	.9908±.0038	.9907±.0037	.9908±.0038	.9907±.0037	—	—	.9902±.0040	.9904±.0046	.9911±.0046	**.9923±.0033**
Amazon_employee_access	.8390±.0045	.8345±.0078	.8391±.0077	.8345±.0078	.8391±.0077	—	—	.8416±.0129	**.8440±.0113**	.8438±.0068	.8197±.0054
Bank_Customer_Churn	.8540±.0044	.8548±.0027	.8544±.0061	.8548±.0027	.8544±.0061	—	—	.8493±.0039	.8472±.0071	.8416±.0063	**.8600±.0062**
Bioresponse	.8716±.0088	.8711±.0072	.8723±.0090	.8711±.0072	.8723±.0090	—	—	.8698±.0067	**.8756±.0101**	.8718±.0102	.8714±.0086
Diabetes130US	.6555±.0014	.6589±.0033	.6586±.0058	.6589±.0033	.6586±.0058	—	—	.6580±.0032	.6548±.0052	.6503±.0058	**.6601±.0048**
E-CommereShippingData	.7366±.0059	.7355±.0072	.7319±.0047	.7355±.0072	.7319±.0047	—	—	.7356±.0061	.7368±.0057	.7360±.0116	**.7369±.0060**
Fitness_Club	.7652±.0134	.7671±.0138	.7644±.0145	.7671±.0138	.7644±.0145	—	—	.7672±.0098	.7681±.0123	.7616±.0143	**.7704±.0228**
GiveMeSomeCredit	.8652±.0016	.8502±.0034	.8570±.0029	.8502±.0034	.8570±.0029	—	—	.8641±.0014	.8606±.0022	.8588±.0030	**.8665±.0011**
HR_Analytics_Job_Change_of_Data_Scientists	.7993±.0061	.7986±.0067	.7977±.0075	.7986±.0067	.7977±.0075	—	—	.7955±.0090	.7959±.0067	.7894±.0066	**.8005±.0077**
Is-this-a-good-customer	.7050±.0360	**.7090±.0167**	**.7090±.0173**	**.7090±.0167**	**.7090±.0173**	—	—	.7014±.0192	.6889±.0235	.6959±.0101	**.7090±.0182**
Marketing_Campaign	**.9070±.0270**	.9030±.0297	.9052±.0291	.9030±.0297	.9052±.0291	—	—	.8942±.0300	.9053±.0281	.8910±.0293	.9047±.0255
NATICUSdroid	.9861±.0043	.9860±.0046	**.9863±.0040**	.9860±.0046	**.9863±.0040**	—	—	.9851±.0040	.9845±.0034	.9850±.0041	.9858±.0034
bank-marketing	.7570±.0063	.7569±.0068	.7561±.0077	.7569±.0068	.7561±.0077	—	—	.7559±.0088	.7504±.0073	.7465±.0078	**.7619±.0075**
blood-transfusion-service-center	.6918±.0336	.6932±.0364	.6962±.0358	.6932±.0364	.6962±.0358	—	—	.6863±.0487	.7002±.0417	.6804±.0374	**.7039±.0399**
churn	.9233±.0164	**.9239±.0176**	.9213±.0176	**.9239±.0176**	.9213±.0176	—	—	.9154±.0144	.9204±.0156	.9185±.0081	.9227±.0175
coil2000_insurance_policies	.7219±.0106	.7264±.0035	.7219±.0134	.7264±.0035	.7219±.0134	—	—	.7157±.0080	.7196±.0140	.7003±.0083	**.7309±.0074**
credit-g	.7763±.0447	.7779±.0522	.7803±.0473	.7779±.0522	.7803±.0473	—	—	.7600±.0439	.7670±.0360	.7551±.0340	**.7805±.0374**
credit_card_clients_default	.7786±.0071	**.7797±.0059**	.7786±.0061	**.7797±.0059**	.7786±.0061	—	—	.7735±.0086	.7695±.0089	.7682±.0095	.7795±.0067
customer_satisfaction_in_airline	.9918±.0003	.9920±.0004	.9919±.0002	.9920±.0004	.9919±.0002	—	—	.9929±.0003	.9936±.0002	.9936±.0001	**.9937±.0002**
diabetes	.8030±.0171	.8011±.0265	.8060±.0241	.8011±.0265	.8060±.0241	—	—	.7948±.0225	.7881±.0165	.7972±.0198	**.8076±.0209**
ecom-offers	.6169	.6166	.6141	.6166	.6141	—	—	.6197	.6229	.6191	**.6250**
hazelnut-spread-contaminant-detection	.9774±.0076	**.9795±.0061**	.9781±.0073	**.9795±.0061**	.9781±.0073	—	—	.9737±.0101	.9767±.0087	.9764±.0086	.9735±.0079
heloc	.7870±.0129	**.7898±.0123**	.7894±.0120	**.7898±.0123**	.7894±.0120	—	—	.7827±.0098	.7788±.0081	.7785±.0092	.7882±.0132
homecredit-default	.8183	**.8224**	.8117	**.8224**	.8117	—	—	.8061	.8028	.7909	.8169
homesite-insurance	.9533	.9542	.9540	.9542	.9540	—	—	.9551	.9537	.9542	**.9554**
in_vehicle_coupon_recommendation	.8221±.0067	.8223±.0058	.8214±.0055	.8223±.0058	.8214±.0055	—	—	.8277±.0035	.8310±.0044	**.8325±.0042**	.8278±.0052
jm1	**.7433±.0055**	.7403±.0054	.7393±.0058	.7403±.0054	.7393±.0058	—	—	.7347±.0066	.7330±.0073	.7306±.0111	.7397±.0075
kddcup09_appetency	.8035±.0141	.8076±.0163	.8031±.0122	.8076±.0163	.8031±.0122	—	—	.8035±.0111	.8063±.0187	.7976±.0140	**.8209±.0110**
online_shoppers_intention	**.9267±.0053**	.9257±.0065	.9255±.0069	.9257±.0065	.9255±.0069	—	—	.9232±.0070	.9246±.0066	.9210±.0068	.9260±.0059
polish_companies_bankruptcy	.9551±.0081	.9212±.0087	.9429±.0068	.9212±.0087	.9429±.0068	—	—	.9533±.0097	.9577±.0094	.9555±.0099	**.9593±.0103**
qsar-biodeg	.9391±.0146	**.9414±.0118**	.9366±.0120	**.9414±.0118**	.9366±.0120	—	—	.9244±.0206	.9304±.0158	.9317±.0115	.9309±.0152
seismic-bumps	.7210±.0542	.7106±.0629	.7135±.0674	.7106±.0629	.7135±.0674	—	—	.7193±.0326	.7156±.0400	.7241±.0453	**.7374±.0669**
taiwanese_bankruptcy_prediction	**.9425±.0219**	.9329±.0231	.9353±.0211	.9329±.0231	.9353±.0211	—	—	.9363±.0209	.9416±.0237	.9424±.0265	.9421±.0193

† RF implementations of this method are not directly comparable. XGBoost 3.4.1 warns that `XGBRFClassifier` is deprecated and "does not implement a conventional random forest". CatBoost does not support Random Forest (Bagged) mode.

* Dynamic methods only benefit trees if they're trained deeper than the GBTs' depth 6. Benefit starts to show at depth 12-16, depending on dataset. As such, they've been excluded from the GBT results.